FIAP
Mastering Relational and Non-Relational Database
Análise e Desenvolvimento de Sistemas · Turma 2TDSPF · 2026

ClyvoVet

Banco de Dados Relacional Oracle — Procedimentos, Funções e Trigger de Auditoria em PL/SQL

Sprint 3 · Conversão relacional → JSON manual · Subtotais sem agregação · Score de risco clínico · Auditoria DML

Integrantes (ordem alfabética)

Nome completo	RM
Carlos Alberto Guedes Neto	566022
Eduardo Novaes Mollo	561515
Luan Peixoto Marins Rocha	562258
Mathaus Victor Souza Marcelino	564146
Vinicius Luis Exposito Morassi	563340

Tema compartilhado com os projetos do grupo em **Java Advanced** (ClyvoVet Web) e **Disruptive Architectures: IoT, IoB & IA Generativa** (CLYVO Copilot): plataforma digital para a jornada contínua de saúde de pets, conectando tutores, clínicas e veterinários.

Sumário

1. Visão geral 4

1.1 O que foi entregue 4

1.2 Ambiente de execução 4

2. Modelo de dados 5

2.1 Estrutura (CREATE TABLE e SEQUENCE) 5

2.2 Carga de dados 7

3. Decisões de projeto 8

3.1 JSON sem funções automáticas e sem CREATE TYPE 8

3.2 Quebra de controle no Procedimento 2 8

3.3 Trigger que reaproveita a Função 1 8

3.4 Estratégia de exceções 8

3.5 Portabilidade 8

4. Função 1 — FN_MONTA_JSON 9

4.1 Finalidade e assinatura 9

4.2 Algoritmo 9

4.3 Código 9

4.4 Execução 12

4.5 Exceções tratadas 12

5. Função 2 — FN_CALC_RISCO_PET 14

5.1 Regra de negócio 14

5.2 Código 14

5.3 Execução 16

5.4 Exceções tratadas 16

6. Utilitário — PRC_IMPRIME_CLOB 18

7. Procedimento 1 — PRC_ATENDIMENTOS_JSON 19

7.1 Finalidade 19

7.2 Código 19

7.3 Execução 20

7.4 Exceções tratadas 23

8. Procedimento 2 — PRC_FATURAMENTO_CLINICA 25

8.1 Finalidade e técnica 25

8.2 Código 25

8.3 Execução 26

8.4 Exceções tratadas . . . 27
9. Trigger de auditoria — TRG_AUD_PET . . . 29
9.1 Tabela de auditoria . . . 29
9.2 Código da trigger . . . 29
9.3 Execução . . . 30
10. Conferências finais . . . 32
11. Como executar e mapa de exceções . . . 34
11.1 Execução . . . 34
11.2 Exceções por rotina e como cada uma é provocada . . . 34
11.3 Sobre a Sprint anterior . . . 34
Anexo A — Carga de dados completa . . . 35
Anexo B — Configuração da sessão e limpeza idempotente . . . 38

1. Visão geral

Este documento acompanha o script **2TDSPF_2026_CodigoSql_Integrantes.sql** e descreve a implementação, em Oracle Database, de dois procedimentos, duas funções e uma trigger de auditoria sobre o domínio **ClyvoVet** — o mesmo tema dos trabalhos do grupo em Java e IoT. A ClyvoVet é uma rede de clínicas veterinárias cujos pacientes usam um dispositivo vestível (ESP32 com sensores de temperatura, frequência cardíaca, SpO2 e acelerômetro). Os dados de telemetria alimentam um score de risco clínico; nesta entrega, essa regra de negócio passa a viver dentro do banco, ao lado do cadastro e do faturamento das unidades.

Todo o código foi escrito em PL/SQL puro. Em particular, **nenhuma função built-in de JSON** (JSON_OBJECT, JSON_ARRAY, JSON_VALUE, JSON_QUERY, JSON_TABLE, TO_JSON, JSON_SERIALIZE) e **nenhum recurso de agregação avançada** (ROLLUP, CUBE, GROUPING SETS, GROUPING) foi utilizado — nas rotinas exigidas o grupo também dispensou SUM, AVG e GROUP BY, para que todo somatório seja visível no corpo dos procedimentos.

1.1 O que foi entregue

Objeto	Tipo	Finalidade	Exceções tratadas
`FN_MONTA_JSON`	Função 1	Converte dados relacionais (três listas paralelas: chaves, valores, tipos) em um objeto JSON. Parse, escape e validação numérica manuais; suporta aninhamento.	e_parametro_nulo, e_qtd_divergente, e_tipo_invalido, VALUE_ERROR, OTHERS
`FN_CALC_RISCO_PET`	Função 2	Regra de negócio do projeto IoT: score de risco 0–100 a partir das médias das leituras de sensor, com faixa de referência por espécie.	e_param_invalido, NO_DATA_FOUND, ZERO_DIVIDE, OTHERS
`PRC_ATENDIMENTOS_JSON`	Procedimento 1	JOIN de 6 tabelas → array JSON. Cada linha vira um objeto gerado pela Função 1, com pet e tutor aninhados.	e_parametro_invalido, NO_DATA_FOUND, e_sem_registros, e_falha_conversao, OTHERS
`PRC_FATURAMENTO_CLINICA`	Procedimento 2	Faturamento por (clínica, serviço), subtotal por clínica e total geral, por quebra de controle em cursor ordenado.	VALUE_ERROR / INVALID_NUMBER, e_ano_invalido, e_sem_movimento, OTHERS
`TRG_AUD_PET + AUD_PET`	Trigger	AFTER INSERT OR UPDATE OR DELETE ON PET, por linha. Registra usuário, operação, data/hora e :OLD/:NEW em JSON.	OTHERS → RAISE_APPLICATION_ERROR(-20051)
`PRC_IMPRIME_CLOB`	Utilitário	Imprime um CLOB linha a linha respeitando o limite de 32 767 bytes do DBMS_OUTPUT.	OTHERS

1.2 Ambiente de execução

O script foi executado de ponta a ponta em **Oracle Database 23ai Free** (imagem oficial *gvenzl/oracle-free*) via SQL*Plus, com o parâmetro NLS americano e também com o brasileiro (decimal com vírgula), produzindo o mesmo JSON nos dois casos. Os prints deste documento são a saída real dessas execuções; o mesmo script roda sem alterações no SQL Developer conectado à instância Oracle da FIAP (executar como *script*, F5).

2. Modelo de dados

Oito tabelas: quatro dimensões (TUTOR, CLINICA, VETERINARIO, TIPO_SERVICO), o cadastro de pacientes (PET), a tabela fato ATENDIMENTO, a telemetria LEITURA_SENSOR e a tabela de auditoria AUD_PET. A tabela fato carrega diretamente as duas colunas categóricas exigidas pelo Procedimento 2 (id_clinica e id_tipo_servico) e a medida numérica somada (valor).

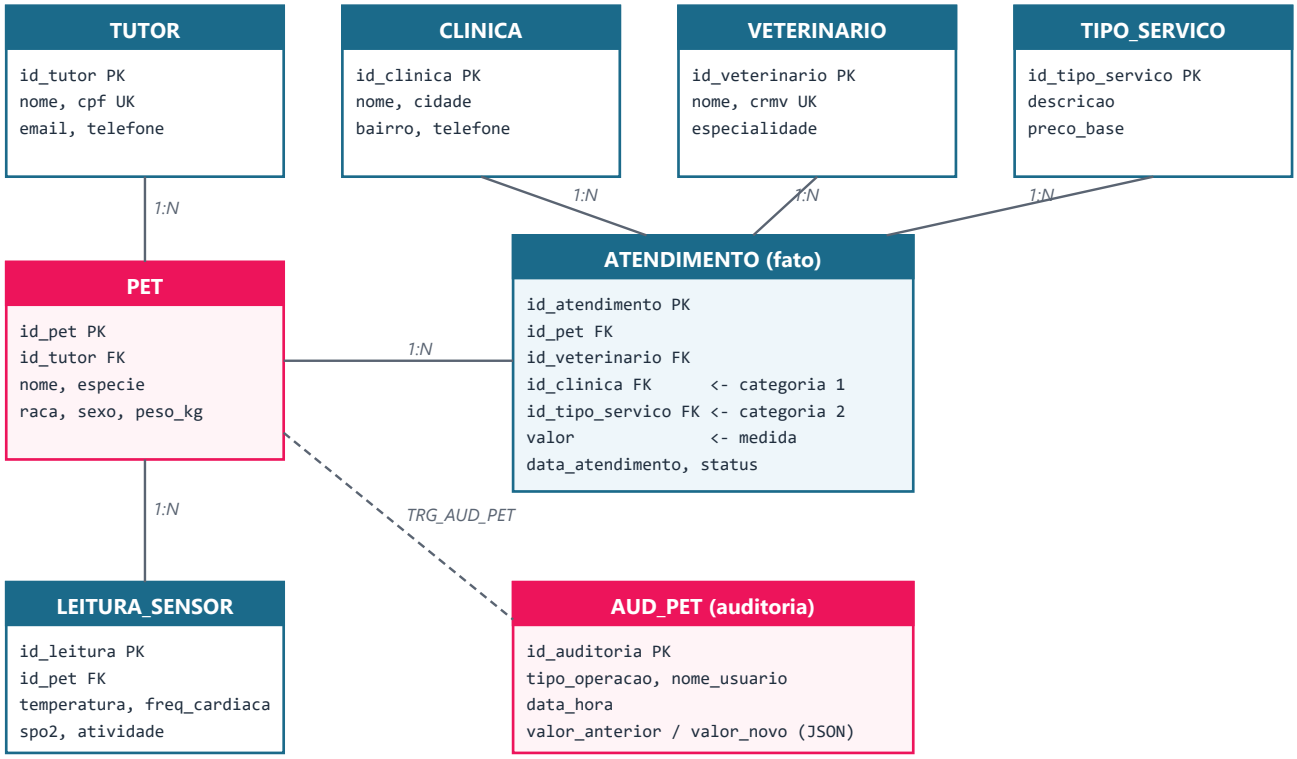


Figura 1 — Modelo de dados. Em destaque (rosa) a tabela auditada e a tabela de auditoria; em azul a tabela fato.

Tabela	Registros	Papel no trabalho
CLINICA	6	1ª categoria do Procedimento 2. A unidade 6 (Campinas) é nova e não tem atendimentos — usada para demonstrar a exceção e_sem_registros.
TIPO_SERVICO	5	2ª categoria do Procedimento 2 (consulta, vacinação, exames, cirurgia, ultrassonografia).
TUTOR	8	Dimensão do JOIN do Procedimento 1.
VETERINARIO	6	Dimensão do JOIN do Procedimento 1.
PET	10 (+2 nos testes)	Tabela auditada pela trigger. Pets 8–10 não têm leituras: o pet 10 demonstra ZERO_DIVIDE na Função 2.
ATENDIMENTO	34	Tabela fato: 30 realizados em 2026 cobrindo as 25 combinações clínica × serviço (+5 repetições), 2 realizados em 2025 e 2 agendados.
LEITURA_SENSOR	20	Insumo da Função 2. Cenários reaproveitados do projeto IoT (Mel com febre gradual, Luna com queda de SpO2).
AUD_PET	6 (gerados nos testes)	Auditoria: usuário, operação, data/hora, :OLD e :NEW em JSON.

2.1 Estrutura (CREATE TABLE e SEQUENCE)

Chaves primárias explícitas na carga inicial (resultados reproduzíveis) e SEQUENCE para os registros criados em tempo de execução — em vez de colunas IDENTITY — por compatibilidade com qualquer versão do

Oracle da FIAP. Constraints nomeadas, CHECK em domínios fechados (espécie, sexo, status, operação) e FKs em todas as ligações.

```
-- ================================================================================
-- [2] ESTRUTURA - TABELAS, CONSTRAINTS E SEQUENCES
--     Ordem: tabelas-pai primeiro (CLINICA, TIPO_SERVICO, TUTOR, VETERINARIO),
--     depois PET, depois as tabelas que dependem de PET (ATENDIMENTO, LEITURA_SENSOR)
--     e por fim a tabela de auditoria.
-- ================================================================================
-- Unidades da rede ClyvoVet. Primeira categoria (nivel externo) do Procedimento 2.
CREATE TABLE CLINICA (
    id_clinica      NUMBER(6)       CONSTRAINT pk_clinica PRIMARY KEY,
    nome            VARCHAR2(80)    NOT NULL,
    cidade          VARCHAR2(60)    NOT NULL,
    bairro          VARCHAR2(60),
    telefone        VARCHAR2(20)
);
-- Catalogo de servicos. Segunda categoria (nivel interno) do Procedimento 2.
CREATE TABLE TIPO_SERVICO (
    id_tipo_servico NUMBER(6)       CONSTRAINT pk_tipo_servico PRIMARY KEY,
    descricao       VARCHAR2(60)    NOT NULL,
    preco_base      NUMBER(10,2)    NOT NULL
                    CONSTRAINT ck_tipo_servico_preco CHECK (preco_base > 0)
);
-- Responsavel pelo pet.
CREATE TABLE TUTOR (
    id_tutor        NUMBER(6)       CONSTRAINT pk_tutor PRIMARY KEY,
    nome            VARCHAR2(100)   NOT NULL,
    cpf             VARCHAR2(14)    CONSTRAINT un_tutor_cpf UNIQUE,
    email           VARCHAR2(100),
    telefone        VARCHAR2(20),
    cidade          VARCHAR2(60)
);
-- Corpo clinico.
CREATE TABLE VETERINARIO (
    id_veterinario  NUMBER(6)       CONSTRAINT pk_veterinario PRIMARY KEY,
    nome            VARCHAR2(100)   NOT NULL,
    crmv            VARCHAR2(20)    NOT NULL
                    CONSTRAINT un_veterinario_crmv UNIQUE,
    especialidade   VARCHAR2(60)
);
-- Paciente. Tabela auditada pela trigger TRG_AUD_PET.
-- ESPECIE guarda um codigo (CAO/GATO) para a Funcao 2 escolher a faixa de referencia.
CREATE TABLE PET (
    id_pet          NUMBER(6)       CONSTRAINT pk_pet PRIMARY KEY,
    id_tutor        NUMBER(6)       NOT NULL
                    CONSTRAINT fk_pet_tutor REFERENCES TUTOR (id_tutor),
    nome            VARCHAR2(60)    NOT NULL,
    especie         VARCHAR2(10)    NOT NULL
                    CONSTRAINT ck_pet_especie CHECK (especie IN ('CAO', 'GATO')),
    raca            VARCHAR2(60),
    sexo            CHAR(1)         CONSTRAINT ck_pet_sexo CHECK (sexo IN ('M', 'F')),
    data_nascimento DATE,
    peso_kg         NUMBER(6,2)     CONSTRAINT ck_pet_peso CHECK (peso_kg > 0)
);
-- TABELA FATO do Procedimento 2: duas colunas categoricas (id_clinica, id_tipo_servico)
-- e uma coluna numerica a ser somada (valor). Tambem e o centro do JOIN do Procedimento 1.
CREATE TABLE ATENDIMENTO (
    id_atendimento   NUMBER(8)      CONSTRAINT pk_atendimento PRIMARY KEY,
    id_pet           NUMBER(6)      NOT NULL
                     CONSTRAINT fk_atend_pet REFERENCES PET (id_pet),
    id_veterinario   NUMBER(6)      NOT NULL
                     CONSTRAINT fk_atend_veterinario REFERENCES VETERINARIO (id_veterinario),
    id_clinica       NUMBER(6)      NOT NULL
                     CONSTRAINT fk_atend_clinica REFERENCES CLINICA (id_clinica),
    id_tipo_servico  NUMBER(6)      NOT NULL
                     CONSTRAINT fk_atend_tipo_servico REFERENCES TIPO_SERVICO (id_tipo_servico),
    data_atendimento DATE           NOT NULL,
    valor            NUMBER(10,2)   NOT NULL
                     CONSTRAINT ck_atend_valor CHECK (valor >= 0),
    status           VARCHAR2(10)   DEFAULT 'REALIZADO' NOT NULL
                     CONSTRAINT ck_atend_status CHECK (status IN ('REALIZADO', 'AGENDADO', 'CANCELADO'))
);
-- Telemetria do dispositivo vestivel (ESP32 com DHT22 / MAX30102 / MPU6050 do projeto IoT).
-- Insumo da Funcao 2. ATIVIDADE e um indice 0..100 derivado do acelerometro.
CREATE TABLE LEITURA_SENSOR (
    id_leitura      NUMBER(10)      CONSTRAINT pk_leitura PRIMARY KEY,
    id_pet          NUMBER(6)       NOT NULL
```

```
                    CONSTRAINT fk_leitura_pet REFERENCES PET (id_pet),
    dispositivo     VARCHAR2(30)    NOT NULL,
    data_hora       DATE            NOT NULL,
    temperatura     NUMBER(4,1)     NOT NULL,   -- graus Celsius
    freq_cardiaca   NUMBER(4)       NOT NULL,   -- batimentos por minuto
    spo2            NUMBER(5,1)     NOT NULL,   -- saturacao de oxigenio (%)
    atividade       NUMBER(5,1)     NOT NULL    -- indice de atividade 0..100
);
-- Tabela de AUDITORIA exigida pelo trabalho: usuario, operacao, data/hora,
-- valores anteriores (:OLD) e valores novos (:NEW). OLD/NEW sao gravados como JSON
-- gerado pela Funcao 1, o que reaproveita a conversao manual em mais um ponto do projeto.
CREATE TABLE AUD_PET (
    id_auditoria    NUMBER(10)      CONSTRAINT pk_aud_pet PRIMARY KEY,
    nome_tabela     VARCHAR2(30)    NOT NULL,
    id_registro     NUMBER(6),                  -- chave do registro afetado em PET
    tipo_operacao   VARCHAR2(10)    NOT NULL
                    CONSTRAINT ck_aud_pet_operacao CHECK (tipo_operacao IN ('INSERT', 'UPDATE', 'DELETE')),
    nome_usuario    VARCHAR2(100)   NOT NULL,   -- usuario Oracle (USER)
    usuario_so      VARCHAR2(100),              -- usuario do sistema operacional do cliente
    maquina         VARCHAR2(100),              -- host de origem da sessao
    data_hora       TIMESTAMP       NOT NULL,
    valor_anterior  CLOB,                       -- JSON com :OLD (nulo no INSERT)
    valor_novo      CLOB                        -- JSON com :NEW (nulo no DELETE)
);
-- Sequences para os registros criados em tempo de execucao. A carga inicial usa ids
-- explicitos (1..N) para que os resultados dos testes sejam reproduziveis; por isso
-- SEQ_PET comeca em 11.
CREATE SEQUENCE SEQ_PET     START WITH 11 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE SEQ_AUD_PET START WITH 1  INCREMENT BY 1 NOCACHE;
```

2.2 Carga de dados

Todas as tabelas recebem pelo menos 5 registros (o mínimo exigido). Os INSERTs completos estão no Anexo A; a contagem final por tabela aparece no capítulo 9. A carga foi desenhada para tornar cada rotina demonstrável: grade 5 × 5 completa para o Procedimento 2, uma clínica sem movimento, pets com e sem leituras e leituras que produzem as quatro classificações de risco.

3. Decisões de projeto

3.1 JSON sem funções automáticas e sem CREATE TYPE

A Função 1 recebe **três strings delimitadas** (chaves, valores e tipos) em vez de uma coleção criada com CREATE TYPE. Motivos: (1) não depende de privilégio para criar tipos no schema; (2) pode ser chamada tanto de PL/SQL quanto de um SELECT; (3) continua genérica — qualquer linha de qualquer tabela vira JSON. O tipo **J** (JSON já montado) permite aninhar objetos, e é assim que o Procedimento 1 embute *pet* e *tutor* em cada atendimento e a trigger serializa :OLD/:NEW.

Toda a conversão é manual: parse das listas com INSTR/SUBSTR, escape caractere a caractere (\, ", \n, \r, \t, \b, \f), validação numérica dígito a dígito e normalização para o padrão JSON (*.5 → 0.5*, *5. → 5*, *007 → 7*, vírgula → ponto — independe do NLS da sessão). Também não se usa LISTAGG nem XMLELEMENT para montar o array: a concatenação acontece no laço da procedure.

3.2 Quebra de controle no Procedimento 2

Um único cursor ordenado por *id_clinica, id_tipo_servico* (a hierarquia das categorias) e três acumuladores PL/SQL: *v_soma_combo*, *v_soma_clinica* e *v_total_geral*. A cada mudança de combinação a linha de detalhe é impressa; a cada mudança de clínica, o subtotal; ao final, o total geral. Nas linhas de subtotal e total geral as colunas de agrupamento ficam em branco (nulas), como no modelo do enunciado.

3.3 Trigger que reaproveita a Função 1

Os campos *valor_anterior* e *valor_novo* de AUD_PET são JSON gerados a partir dos escalares :OLD/:NEW. Como FN_MONTA_JSON não consulta a tabela PET, não existe risco de ORA-04091 (*mutating table*). A trigger é criada depois da carga inicial, de modo que a auditoria contenha apenas as operações do roteiro de testes. Falha ao auditar não é engolida: vira ORA-20051 e desfaz a operação.

3.4 Estratégia de exceções

Cada rotina declara pelo menos três exceções nomeadas ou próprias **além** de WHEN OTHERS, e o roteiro de testes provoca cada uma com um parâmetro específico. As funções imprimem o motivo via DBMS_OUTPUT e devolvem NULL (Função 1) ou -1 (Função 2); as procedures imprimem e encerram normalmente. Assim a exceção tratada aparece no print e a execução do script nunca é interrompida.

3.5 Portabilidade

Literais sem acento (o script roda idêntico em qualquer charset de banco/cliente), formatos de data e número sempre explícitos, SEQUENCE em vez de IDENTITY, e um bloco de limpeza idempotente que ignora apenas os erros "objeto não existe" (ORA-00942, -02289, -04080, -04043) e propaga qualquer outro.

4. Função 1 — FN_MONTA_JSON

4.1 Finalidade e assinatura

Recebe dados relacionais e devolve uma string JSON. As três listas são paralelas: a i-ésima chave recebe o i-ésimo valor com o i-ésimo tipo. Valor vazio em qualquer tipo produz o literal *null*.

```
FN_MONTA_JSON(p_chaves, p_valores, p_tipos, p_sep DEFAULT '|') RETURN VARCHAR2
FN_MONTA_JSON('id|nome|peso_kg', '1|Rex|32.5', 'N|S|N')
   => {"id":1,"nome":"Rex","peso_kg":32.5}
Tipos:  S = string (aspas + escape)   N = numero   B = booleano (true/false)   J = JSON pronto (aninhamento)
```

4.2 Algoritmo

- **Validação**: listas de chaves e tipos e o separador são obrigatórios (e_parametro_nulo); as três listas precisam ter o mesmo número de itens (e_qtd_divergente).
- **Parse**: *contar_itens* conta separadores com INSTR; *item(lista, i)* extrai o i-ésimo elemento com INSTR/SUBSTR. Item vazio → NULL.
- **Chave**: sempre entre aspas, escapada. Chave em branco → e_parametro_nulo.
- **Valor**: S → *escapar()* percorre caractere a caractere; N → *numero_json()* valida sinal, dígitos e um único ponto e normaliza; B → *booleano_json()*; J → inserido sem alteração; tipo fora de S/N/B/J → e_tipo_invalido; valor incompatível → VALUE_ERROR.
- **Montagem**: pares separados por vírgula dentro de chaves { }.

4.3 Código

```
-- ================================================================================
-- [4] FUNCAO 1 - FN_MONTA_JSON
--     Recebe dados relacionais na forma de tres listas paralelas delimitadas
--     (chaves, valores e tipos) e devolve um objeto JSON como string.
--
--     Exemplo:  FN_MONTA_JSON('id|nome|peso_kg', '1|Rex|32.5', 'N|S|N')
--               => {"id":1,"nome":"Rex","peso_kg":32.5}
--
--     Tipos aceitos:  S = string (com aspas e escape)   N = numero (sem aspas)
--                     B = booleano (true/false)          J = JSON ja montado (permite aninhar)
--     Valor vazio em qualquer tipo => null.
--
--     Decisoes de projeto:
--       * Listas delimitadas em vez de CREATE TYPE: nao depende de privilegio para criar
--         tipos no schema e a funcao pode ser chamada tanto do PL/SQL quanto do SQL.
--       * Toda a conversao (parse das listas, escape de caracteres, validacao numerica)
--         e feita com laços e SUBSTR/INSTR. Nenhuma funcao built-in de JSON e usada.
--       * Aceita virgula ou ponto como separador decimal no valor de entrada e sempre
--         emite ponto, como exige o padrao JSON, independentemente do NLS da sessao.
--     Excecoes tratadas: e_parametro_nulo, e_qtd_divergente, e_tipo_invalido, VALUE_ERROR, OTHERS.
-- ================================================================================
CREATE OR REPLACE FUNCTION FN_MONTA_JSON (
    p_chaves   IN VARCHAR2,               -- ex.: 'id|nome|peso_kg'
    p_valores  IN VARCHAR2,               -- ex.: '1|Rex|32.5'
    p_tipos    IN VARCHAR2,               -- ex.: 'N|S|N'
    p_sep      IN VARCHAR2 DEFAULT '|'    -- separador das listas
) RETURN VARCHAR2
IS
    -- excecoes proprias ------------------------------------------------------
    e_parametro_nulo    EXCEPTION;   -- lista obrigatoria vazia ou chave em branco
    e_qtd_divergente    EXCEPTION;   -- listas com quantidades diferentes de itens
    e_tipo_invalido     EXCEPTION;   -- tipo fora de S / N / B / J
    -- variaveis de trabalho --------------------------------------------------
    v_qtd_chaves   PLS_INTEGER;
    v_qtd_valores  PLS_INTEGER;
    v_qtd_tipos    PLS_INTEGER;
    v_chave        VARCHAR2(4000);
    v_valor        VARCHAR2(4000);
    v_tipo         VARCHAR2(10);
    v_detalhe      VARCHAR2(400);    -- complemento da mensagem de erro
```

```
    v_json         VARCHAR2(32767);
    -- Conta os itens de uma lista delimitada: numero de separadores + 1.
    FUNCTION contar_itens(p_lista VARCHAR2) RETURN PLS_INTEGER IS
        v_qtd PLS_INTEGER := 1;
        v_pos PLS_INTEGER := 1;
    BEGIN
        IF p_lista IS NULL THEN
            RETURN 1;                               -- lista vazia = um unico item nulo
        END IF;
        LOOP
            v_pos := INSTR(p_lista, p_sep, v_pos);
            EXIT WHEN NVL(v_pos, 0) = 0;
            v_qtd := v_qtd + 1;
            v_pos := v_pos + LENGTH(p_sep);
        END LOOP;
        RETURN v_qtd;
    END contar_itens;
    -- Devolve o i-esimo item da lista (item vazio => NULL).
    FUNCTION item(p_lista VARCHAR2, p_i PLS_INTEGER) RETURN VARCHAR2 IS
        v_ini PLS_INTEGER;
        v_fim PLS_INTEGER;
    BEGIN
        IF p_lista IS NULL THEN
            RETURN NULL;
        END IF;
        IF p_i = 1 THEN
            v_ini := 1;
        ELSE
            v_ini := INSTR(p_lista, p_sep, 1, p_i - 1) + LENGTH(p_sep);
        END IF;
        v_fim := INSTR(p_lista, p_sep, 1, p_i);    -- separador seguinte (0 = ultimo item)
        IF v_fim = 0 THEN
            RETURN SUBSTR(p_lista, v_ini);
        END IF;
        RETURN SUBSTR(p_lista, v_ini, v_fim - v_ini);
    END item;
    -- Escapa, caractere a caractere, o que nao pode aparecer cru dentro de aspas JSON.
    FUNCTION escapar(p_txt VARCHAR2) RETURN VARCHAR2 IS
        v_out VARCHAR2(32767);
        v_c   VARCHAR2(4);
    BEGIN
        FOR i IN 1 .. NVL(LENGTH(p_txt), 0) LOOP
            v_c := SUBSTR(p_txt, i, 1);
            v_out := v_out ||
                CASE v_c
                    WHEN '\'     THEN '\\'
                    WHEN '"'     THEN '\"'
                    WHEN CHR(10) THEN '\n'
                    WHEN CHR(13) THEN '\r'
                    WHEN CHR(9)  THEN '\t'
                    WHEN CHR(8)  THEN '\b'
                    WHEN CHR(12) THEN '\f'
                    ELSE v_c
                END;
        END LOOP;
        RETURN v_out;
    END escapar;
    -- Valida e normaliza um numero para o padrao JSON: sinal opcional, digitos e no
    -- maximo um ponto decimal. Aceita virgula na entrada. Levanta VALUE_ERROR se invalido.
    FUNCTION numero_json(p_txt VARCHAR2) RETURN VARCHAR2 IS
        v_txt     VARCHAR2(100) := REPLACE(TRIM(p_txt), ',', '.');
        v_sinal   VARCHAR2(1)   := '';
        v_c       VARCHAR2(1);
        v_pontos  PLS_INTEGER := 0;
        v_digitos PLS_INTEGER := 0;
    BEGIN
        IF SUBSTR(v_txt, 1, 1) IN ('-', '+') THEN
            IF SUBSTR(v_txt, 1, 1) = '-' THEN
                v_sinal := '-';
            END IF;
            v_txt := SUBSTR(v_txt, 2);
        END IF;
        FOR i IN 1 .. NVL(LENGTH(v_txt), 0) LOOP
            v_c := SUBSTR(v_txt, i, 1);
            IF v_c BETWEEN '0' AND '9' THEN
                v_digitos := v_digitos + 1;
            ELSIF v_c = '.' THEN
                v_pontos := v_pontos + 1;
            ELSE
                RAISE VALUE_ERROR;                  -- caractere que nao pertence a um numero
```

```
            END IF;
        END LOOP;
        IF v_digitos = 0 OR v_pontos > 1 THEN
            RAISE VALUE_ERROR;
        END IF;
        -- JSON nao aceita ".5", "5." nem zeros a esquerda ("007")
        IF SUBSTR(v_txt, 1, 1) = '.' THEN
            v_txt := '0' || v_txt;
        END IF;
        IF SUBSTR(v_txt, -1) = '.' THEN
            v_txt := SUBSTR(v_txt, 1, LENGTH(v_txt) - 1);
        END IF;
        WHILE LENGTH(v_txt) > 1 AND SUBSTR(v_txt, 1, 1) = '0' AND SUBSTR(v_txt, 2, 1) <> '.' LOOP
            v_txt := SUBSTR(v_txt, 2);
        END LOOP;
        RETURN v_sinal || v_txt;
    END numero_json;
    -- Converte representacoes usuais de booleano para true/false.
    FUNCTION booleano_json(p_txt VARCHAR2) RETURN VARCHAR2 IS
        v_txt VARCHAR2(20) := UPPER(TRIM(p_txt));
    BEGIN
        IF v_txt IN ('S', 'SIM', 'TRUE', 'T', 'V', '1') THEN
            RETURN 'true';
        ELSIF v_txt IN ('N', 'NAO', 'FALSE', 'F', '0') THEN
            RETURN 'false';
        END IF;
        RAISE VALUE_ERROR;
    END booleano_json;
BEGIN
    -- 1) validacao dos parametros
    IF p_chaves IS NULL OR p_tipos IS NULL OR p_sep IS NULL THEN
        v_detalhe := 'as listas de chaves e de tipos e o separador sao obrigatorios';
        RAISE e_parametro_nulo;
    END IF;
    v_qtd_chaves  := contar_itens(p_chaves);
    v_qtd_valores := contar_itens(p_valores);
    v_qtd_tipos   := contar_itens(p_tipos);
    IF v_qtd_chaves <> v_qtd_valores OR v_qtd_chaves <> v_qtd_tipos THEN
        RAISE e_qtd_divergente;
    END IF;
    -- 2) montagem do objeto, par a par
    v_json := '{';
    FOR i IN 1 .. v_qtd_chaves LOOP
        v_chave := TRIM(item(p_chaves, i));
        v_valor := item(p_valores, i);
        v_tipo  := UPPER(TRIM(item(p_tipos, i)));
        IF v_chave IS NULL THEN
            v_detalhe := 'chave em branco na posicao ' || i;
            RAISE e_parametro_nulo;
        END IF;
        IF v_tipo IS NULL OR v_tipo NOT IN ('S', 'N', 'B', 'J') THEN
            RAISE e_tipo_invalido;
        END IF;
        IF i > 1 THEN
            v_json := v_json || ',';
        END IF;
        v_json := v_json || '"' || escapar(v_chave) || '":';
        IF v_valor IS NULL THEN
            v_json := v_json || 'null';
        ELSIF v_tipo = 'S' THEN
            v_json := v_json || '"' || escapar(v_valor) || '"';
        ELSIF v_tipo = 'N' THEN
            v_json := v_json || numero_json(v_valor);
        ELSIF v_tipo = 'B' THEN
            v_json := v_json || booleano_json(v_valor);
        ELSE                                        -- 'J': JSON pronto (objeto/array aninhado)
            v_json := v_json || v_valor;
        END IF;
    END LOOP;
    v_json := v_json || '}';
    RETURN v_json;
EXCEPTION
    WHEN e_parametro_nulo THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON] Erro tratado (e_parametro_nulo): ' || v_detalhe || '.');
        RETURN NULL;
    WHEN e_qtd_divergente THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON] Erro tratado (e_qtd_divergente): ' || v_qtd_chaves || ' chave(s), '
                             || v_qtd_valores || ' valor(es) e ' || v_qtd_tipos || ' tipo(s). As listas devem ter o
        mesmo tamanho.');
        RETURN NULL;
```

```
    WHEN e_tipo_invalido THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON] Erro tratado (e_tipo_invalido): tipo "' || v_tipo || '" invalido para a
        chave "'
                            || v_chave || '". Use S, N, B ou J.');
        RETURN NULL;
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON] Erro tratado (VALUE_ERROR): valor "' || v_valor || '" incompativel com o
         tipo '
                            || v_tipo || ' na chave "' || v_chave || '".');
        RETURN NULL;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON] Erro inesperado ' || SQLCODE || ': ' || SQLERRM);
        RETURN NULL;
END FN_MONTA_JSON;
/
SHOW ERRORS
```

4.4 Execução

Bloco 10.1: objeto com string, número (entrada com vírgula decimal), booleano, valor nulo, aspas e barra invertida escapadas; em seguida um objeto com dois objetos aninhados (tipo J). Bloco 10.2: a mesma função usada dentro de um SELECT sobre a tabela PET.

```
-- 10.1 FUNCAO 1 - execucao normal (PL/SQL): objeto simples, escape e aninhamento
-- 10.2 FUNCAO 1 - uso direto em SQL sobre a tabela PET
```

Saida do script | 10.1 FN_MONTA_JSON - execucao normal

```
{"id":1,"nome":"Rex","especie":"CAO","peso_kg":32.5,"castrado":true,"alergias":null,"observacao":"Gosta de
      \"petiscos\" e corre em zigue\\zague"}
{"pet":{"id":1,"nome":"Rex"},"tutor":{"id":1,"nome":"Ana Paula Ribeiro"},"ativo":true}

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 1 — FN_MONTA_JSON: execução normal (escape, tipos e aninhamento).

Saida do script | 10.2 FN_MONTA_JSON - uso em SQL

```
    ID_PET PET_JSON
---------- --------------------------------------------------------------------------------
         1 {"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","peso_kg":32.5}
         2 {"id":2,"nome":"Mel","especie":"CAO","raca":"Golden Retriever","peso_kg":28}
         3 {"id":3,"nome":"Luna","especie":"GATO","raca":"Siames","peso_kg":4.2}
         4 {"id":4,"nome":"Thor","especie":"CAO","raca":"Pastor Alemao","peso_kg":38}
         5 {"id":5,"nome":"Nina","especie":"GATO","raca":"Persa","peso_kg":3.8}

5 rows selected.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 2 — FN_MONTA_JSON chamada em SQL para as cinco primeiras linhas de PET.

4.5 Exceções tratadas

Quatro chamadas inválidas, cada uma cai em um handler diferente; a função imprime o motivo e devolve NULL.

```
-- 10.3 FUNCAO 1 - excecoes tratadas (a funcao imprime o motivo e devolve NULL)
```

Saida do script | 10.3 FN_MONTA_JSON - excecoes tratadas

```
[FN_MONTA_JSON] Erro tratado (e_qtd_divergente): 2 chave(s), 1 valor(es) e 2 tipo(s). As listas devem ter o mesmo
      tamanho.
Retorno: NULL
[FN_MONTA_JSON] Erro tratado (e_tipo_invalido): tipo "X" invalido para a chave "nome". Use S, N, B ou J.
Retorno: NULL
[FN_MONTA_JSON] Erro tratado (VALUE_ERROR): valor "trinta" incompativel com o tipo N na chave "peso_kg".
Retorno: NULL
[FN_MONTA_JSON] Erro tratado (e_parametro_nulo): as listas de chaves e de tipos e o separador sao obrigatorios.
Retorno: NULL

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 3 — FN_MONTA_JSON: e_qtd_divergente, e_tipo_invalido, VALUE_ERROR e e_parametro_nulo.

5. Função 2 — FN_CALC_RISCO_PET

5.1 Regra de negócio

Substitui a lógica de triagem do CLYVO Copilot (projeto IoT): a partir das leituras do dispositivo vestível, calcula um score de risco de 0 a 100 e o classifica. A espécie define a faixa de referência; as médias são somadas manualmente em um laço; cada sinal vital gera um sub-score proporcional ao quanto a média escapa da faixa normal; o score final é a média ponderada dos sub-scores.

Sinal vital	Cão (faixa normal)	Gato (faixa normal)	Sensibilidade (sub-score 100)	Peso
Temperatura (°C)	37,5 – 39,2	38,0 – 39,5	0,8 °C fora da faixa	40%
Freq. cardíaca (bpm)	60 – 140	120 – 220	40 bpm fora da faixa	25%
SpO2 (%)	≥ 95	≥ 95	5 pontos abaixo de 95	20%
Atividade (0–100)	≥ 40	≥ 30	25 pontos abaixo do mínimo	15%

Classificação: score < 25 **BAIXO** · < 50 **MODERADO** · < 75 **ALTO** · ≥ 75 **CRÍTICO**. A função devolve o score numérico (para uso em SQL e em outras rotinas) e imprime uma linha de diagnóstico com as médias e a classificação, para conferência do cálculo.

5.2 Código

```
-- ================================================================================
-- [5] FUNCAO 2 - FN_CALC_RISCO_PET
--     Substitui a logica de triagem do projeto IoT (CLYVO Copilot): calcula um score de
--     risco clinico 0..100 para um pet a partir das leituras dos sensores vestiveis.
--
--     Regra de negocio:
--       1. A especie define a faixa de referencia (cao x gato).
--       2. Media de temperatura, frequencia cardiaca, SpO2 e atividade, somadas manualmente.
--       3. Cada sinal vital gera um sub-score 0..100 proporcional ao quanto a media
--          escapa da faixa normal (sensibilidade por sinal).
--       4. Score final = 40% temperatura + 25% frequencia cardiaca + 20% SpO2 + 15% atividade.
--       5. Classificacao: < 25 BAIXO | < 50 MODERADO | < 75 ALTO | >= 75 CRITICO.
--     Retorna -1 quando uma excecao e tratada.
--     Excecoes tratadas: e_param_invalido, NO_DATA_FOUND, ZERO_DIVIDE, OTHERS.
-- ================================================================================
CREATE OR REPLACE FUNCTION FN_CALC_RISCO_PET (p_pet_id IN NUMBER) RETURN NUMBER
IS
    e_param_invalido  EXCEPTION;              -- id nulo ou nao positivo
    v_nome        PET.nome%TYPE;
    v_especie     PET.especie%TYPE;
    -- acumuladores (somatorio manual)
    v_qtd         PLS_INTEGER := 0;
    v_soma_temp   NUMBER := 0;
    v_soma_fc     NUMBER := 0;
    v_soma_spo2   NUMBER := 0;
    v_soma_ativ   NUMBER := 0;
    -- medias
    v_med_temp    NUMBER;
    v_med_fc      NUMBER;
    v_med_spo2    NUMBER;
    v_med_ativ    NUMBER;
    -- faixas de referencia por especie
    v_temp_min    NUMBER;
    v_temp_max    NUMBER;
    v_fc_min      NUMBER;
    v_fc_max      NUMBER;
    v_spo2_min    NUMBER := 95;
    v_ativ_min    NUMBER;
    -- sub-scores e resultado
    v_sub_temp    NUMBER;
    v_sub_fc      NUMBER;
    v_sub_spo2    NUMBER;
    v_sub_ativ    NUMBER;
    v_score       NUMBER;
    -- Sub-score 0..100: distancia da media ate a faixa normal x fator de sensibilidade.
    FUNCTION sub_score(p_media NUMBER, p_min NUMBER, p_max NUMBER, p_fator NUMBER) RETURN NUMBER IS
```

```
        v_desvio NUMBER := 0;
    BEGIN
        IF p_media < p_min THEN
            v_desvio := p_min - p_media;
        ELSIF p_max IS NOT NULL AND p_media > p_max THEN
            v_desvio := p_media - p_max;
        END IF;
        RETURN LEAST(100, v_desvio * p_fator);
    END sub_score;
    FUNCTION classificar(p_score NUMBER) RETURN VARCHAR2 IS
    BEGIN
        IF    p_score < 25 THEN RETURN 'BAIXO';
        ELSIF p_score < 50 THEN RETURN 'MODERADO';
        ELSIF p_score < 75 THEN RETURN 'ALTO';
        ELSE                    RETURN 'CRITICO';
        END IF;
    END classificar;
    FUNCTION fmt(p_num NUMBER) RETURN VARCHAR2 IS
    BEGIN
        RETURN TO_CHAR(p_num, 'FM9990.0', 'NLS_NUMERIC_CHARACTERS=''.,''');
    END fmt;
BEGIN
    -- 1) validacao do parametro
    IF p_pet_id IS NULL OR p_pet_id <= 0 THEN
        RAISE e_param_invalido;
    END IF;
    -- 2) especie define a faixa de referencia (NO_DATA_FOUND se o pet nao existir)
    SELECT nome, especie
      INTO v_nome, v_especie
      FROM PET
     WHERE id_pet = p_pet_id;
    IF v_especie = 'GATO' THEN
        v_temp_min := 38.0;  v_temp_max := 39.5;
        v_fc_min   := 120;   v_fc_max   := 220;
        v_ativ_min := 30;
    ELSE
        v_temp_min := 37.5;  v_temp_max := 39.2;
        v_fc_min   := 60;    v_fc_max   := 140;
        v_ativ_min := 40;
    END IF;
    -- 3) somatorio manual das leituras (sem SUM / AVG)
    FOR r IN (SELECT temperatura, freq_cardiaca, spo2, atividade
                FROM LEITURA_SENSOR
               WHERE id_pet = p_pet_id) LOOP
        v_qtd       := v_qtd + 1;
        v_soma_temp := v_soma_temp + r.temperatura;
        v_soma_fc   := v_soma_fc   + r.freq_cardiaca;
        v_soma_spo2 := v_soma_spo2 + r.spo2;
        v_soma_ativ := v_soma_ativ + r.atividade;
    END LOOP;
    -- 4) medias (ZERO_DIVIDE se o pet nao possui leituras)
    v_med_temp := v_soma_temp / v_qtd;
    v_med_fc   := v_soma_fc   / v_qtd;
    v_med_spo2 := v_soma_spo2 / v_qtd;
    v_med_ativ := v_soma_ativ / v_qtd;
    -- 5) sub-scores: fator calibrado para que o desvio "grave" de cada sinal valha 100
    v_sub_temp := sub_score(v_med_temp, v_temp_min, v_temp_max, 125);  -- 0,8 C fora da faixa
    v_sub_fc   := sub_score(v_med_fc,   v_fc_min,   v_fc_max,   2.5);  -- 40 bpm fora da faixa
    v_sub_spo2 := sub_score(v_med_spo2, v_spo2_min, NULL,       20);   -- 5 pontos abaixo de 95%
    v_sub_ativ := sub_score(v_med_ativ, v_ativ_min, NULL,       4);    -- 25 pontos abaixo do minimo
    -- 6) score ponderado
    v_score := ROUND(0.40 * v_sub_temp + 0.25 * v_sub_fc + 0.20 * v_sub_spo2 + 0.15 * v_sub_ativ, 1);
    -- linha de diagnostico para conferencia do calculo
    DBMS_OUTPUT.PUT_LINE('[FN_CALC_RISCO_PET] ' || RPAD(v_nome, 6) || ' (' || RPAD(v_especie, 4) || ') '
                         || v_qtd || ' leituras | medias: temp ' || fmt(v_med_temp) || 'C, fc ' || fmt(v_med_fc)
                         || ' bpm, spo2 ' || fmt(v_med_spo2) || '%, ativ ' || fmt(v_med_ativ)
                         || ' | score ' || fmt(v_score) || ' -> ' || classificar(v_score));
    RETURN v_score;
EXCEPTION
    WHEN e_param_invalido THEN
        DBMS_OUTPUT.PUT_LINE('[FN_CALC_RISCO_PET] Erro tratado (e_param_invalido): o id do pet deve ser um numero
        positivo. Recebido: '
                             || NVL(TO_CHAR(p_pet_id), 'NULL'));
        RETURN -1;
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('[FN_CALC_RISCO_PET] Erro tratado (NO_DATA_FOUND): nao existe pet com id ' || p_pet_id ||
         '.');
        RETURN -1;
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('[FN_CALC_RISCO_PET] Erro tratado (ZERO_DIVIDE): o pet ' || v_nome || ' (id ' || p_pet_id
```

```
                            || ') nao possui leituras de sensor; impossivel calcular as medias.');
        RETURN -1;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_CALC_RISCO_PET] Erro inesperado ' || SQLCODE || ': ' || SQLERRM);
        RETURN -1;
END FN_CALC_RISCO_PET;
/
SHOW ERRORS
```

5.3 Execução

Bloco 10.4: score dos sete pets monitorados — Rex saudável (0,0), Mel com febre gradual e queda de atividade (86,6 → CRÍTICO), Luna com queda de SpO2 (64,4 → ALTO), Mia levemente febril (33,0 → MODERADO). Bloco 10.4b: a função usada em um SELECT, ordenando os pets por risco.

```
-- 10.4 FUNCAO 2 - execucao normal: score de risco dos pets monitorados
```

Saida do script | 10.4 FN_CALC_RISCO_PET - execucao normal

```
[FN_CALC_RISCO_PET] Rex    (CAO ) 3 leituras | medias: temp 38.5C, fc 90.0 bpm, spo2 98.0%, ativ 70.0 | score 0.0 ->
       BAIXO
[FN_CALC_RISCO_PET] Mel    (CAO ) 4 leituras | medias: temp 40.3C, fc 165.0 bpm, spo2 91.0%, ativ 12.0 | score 86.6
      -> CRITICO
[FN_CALC_RISCO_PET] Luna   (GATO) 3 leituras | medias: temp 39.9C, fc 235.0 bpm, spo2 89.0%, ativ 5.0 | score 64.4
      -> ALTO
[FN_CALC_RISCO_PET] Thor   (CAO ) 3 leituras | medias: temp 39.4C, fc 110.0 bpm, spo2 97.0%, ativ 60.0 | score 10.0
      -> BAIXO
[FN_CALC_RISCO_PET] Nina   (GATO) 2 leituras | medias: temp 38.7C, fc 155.0 bpm, spo2 98.0%, ativ 50.0 | score 0.0
      -> BAIXO
[FN_CALC_RISCO_PET] Bob    (CAO ) 2 leituras | medias: temp 38.9C, fc 150.0 bpm, spo2 96.0%, ativ 35.0 | score 9.3
      -> BAIXO
[FN_CALC_RISCO_PET] Mia    (GATO) 3 leituras | medias: temp 40.0C, fc 180.0 bpm, spo2 93.0%, ativ 40.0 | score 33.0
      -> MODERADO

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 4 — FN_CALC_RISCO_PET: diagnóstico dos pets 1 a 7 com as quatro classificações.

Saida do script | 10.4b FN_CALC_RISCO_PET - uso em SQL

```
    ID_PET NOME       ESPECIE  SCORE_RISCO
---------- ---------- -------- -----------
         2 Mel        CAO             86.6
         3 Luna       GATO            64.4
         7 Mia        GATO            33.0
         1 Rex        CAO              0.0

4 rows selected.

[FN_CALC_RISCO_PET] Rex    (CAO ) 3 leituras | medias: temp 38.5C, fc 90.0 bpm, spo2 98.0%, ativ 70.0 | score 0.0 ->
       BAIXO
[FN_CALC_RISCO_PET] Mel    (CAO ) 4 leituras | medias: temp 40.3C, fc 165.0 bpm, spo2 91.0%, ativ 12.0 | score 86.6
      -> CRITICO
[FN_CALC_RISCO_PET] Luna   (GATO) 3 leituras | medias: temp 39.9C, fc 235.0 bpm, spo2 89.0%, ativ 5.0 | score 64.4
      -> ALTO
[FN_CALC_RISCO_PET] Mia    (GATO) 3 leituras | medias: temp 40.0C, fc 180.0 bpm, spo2 93.0%, ativ 40.0 | score 33.0
      -> MODERADO
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 5 — FN_CALC_RISCO_PET usada em SQL (score ao lado do cadastro).

5.4 Exceções tratadas

```
-- 10.5 FUNCAO 2 - excecoes tratadas (a funcao imprime o motivo e devolve -1)
```

Saida do script | 10.5 FN_CALC_RISCO_PET - excecoes tratadas

```
[FN_CALC_RISCO_PET] Erro tratado (NO_DATA_FOUND): nao existe pet com id 999.
Retorno: -1
[FN_CALC_RISCO_PET] Erro tratado (ZERO_DIVIDE): o pet Zeus (id 10) nao possui leituras de sensor; impossivel
      calcular as medias.
Retorno: -1
[FN_CALC_RISCO_PET] Erro tratado (e_param_invalido): o id do pet deve ser um numero positivo. Recebido: -5
Retorno: -1

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 6 — FN_CALC_RISCO_PET: NO_DATA_FOUND (pet 999), ZERO_DIVIDE (pet sem leituras) e e_param_invalido (-5).

6. Utilitário — PRC_IMPRIME_CLOB

DBMS_OUTPUT.PUT_LINE aceita no máximo 32 767 bytes por linha. O Procedimento 1 acumula o array JSON em um CLOB, e este utilitário o imprime respeitando as quebras de linha já existentes e fatiando em blocos de 32 000 caracteres quando não houver quebra. Só usa DBMS_LOB.GETLENGTH / INSTR / SUBSTR (leitura), sem nenhuma função de JSON.

```
-- ================================================================================
-- [6] UTILITARIO - PRC_IMPRIME_CLOB
--     DBMS_OUTPUT.PUT_LINE aceita no maximo 32767 bytes por linha. Este utilitario
--     imprime um CLOB de qualquer tamanho, respeitando as quebras de linha (CHR(10))
--     ja existentes e fatiando em blocos de 32000 caracteres quando nao houver quebra.
-- ================================================================================
CREATE OR REPLACE PROCEDURE PRC_IMPRIME_CLOB (p_texto IN CLOB)
IS
    c_max    CONSTANT PLS_INTEGER := 32000;
    v_tam    PLS_INTEGER;
    v_ini    PLS_INTEGER := 1;
    v_pos    PLS_INTEGER;
    v_qtd    PLS_INTEGER;
    v_linha  VARCHAR2(32767);
BEGIN
    IF p_texto IS NULL OR DBMS_LOB.GETLENGTH(p_texto) = 0 THEN
        DBMS_OUTPUT.PUT_LINE('(vazio)');
        RETURN;
    END IF;
    v_tam := DBMS_LOB.GETLENGTH(p_texto);
    WHILE v_ini <= v_tam LOOP
        v_pos := DBMS_LOB.INSTR(p_texto, CHR(10), v_ini, 1);      -- proxima quebra de linha
        IF v_pos = 0 OR v_pos - v_ini >= c_max THEN
            -- sem quebra a vista (ou linha longa demais): imprime um bloco de tamanho fixo
            v_qtd   := LEAST(c_max, v_tam - v_ini + 1);
            v_linha := DBMS_LOB.SUBSTR(p_texto, v_qtd, v_ini);
            v_ini   := v_ini + v_qtd;
        ELSIF v_pos = v_ini THEN
            v_linha := NULL;                                       -- linha em branco
            v_ini   := v_pos + 1;
        ELSE
            v_linha := DBMS_LOB.SUBSTR(p_texto, v_pos - v_ini, v_ini);
            v_ini   := v_pos + 1;
        END IF;
        DBMS_OUTPUT.PUT_LINE(v_linha);
    END LOOP;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_IMPRIME_CLOB] Erro inesperado ' || SQLCODE || ': ' || SQLERRM);
END PRC_IMPRIME_CLOB;
/
SHOW ERRORS
```

7. Procedimento 1 — PRC_ATENDIMENTOS_JSON

7.1 Finalidade

Faz o JOIN de **seis tabelas** — ATENDIMENTO, CLINICA, VETERINARIO, TIPO_SERVICO, PET e TUTOR — e exibe o resultado como um array JSON (string). Para cada linha do JOIN a procedure monta dois objetos aninhados (pet e tutor) e o objeto do atendimento, todos via FN_MONTA_JSON, e os concatena em um CLOB separados por vírgula. O parâmetro opcional *p_clinica_id* filtra por unidade e é validado com SELECT INTO (NO_DATA_FOUND se não existir).

7.2 Código

```
-- ================================================================================
-- [7] PROCEDURE 1 - PRC_ATENDIMENTOS_JSON
--     Faz o JOIN de 6 tabelas (ATENDIMENTO, CLINICA, VETERINARIO, TIPO_SERVICO, PET e
--     TUTOR) e exibe o resultado como um array JSON (string). Cada linha do JOIN vira um
--     objeto gerado pela Funcao 1; PET e TUTOR entram como objetos aninhados (tipo J).
--     O array e acumulado em um CLOB e impresso por PRC_IMPRIME_CLOB.
--
--     p_clinica_id: NULL = todas as clinicas; informado = filtra e valida a clinica.
--     Excecoes tratadas: e_parametro_invalido, NO_DATA_FOUND, e_sem_registros,
--                        e_falha_conversao, OTHERS.
-- ================================================================================
CREATE OR REPLACE PROCEDURE PRC_ATENDIMENTOS_JSON (p_clinica_id IN NUMBER DEFAULT NULL)
IS
    e_parametro_invalido  EXCEPTION;      -- id de clinica nao positivo
    e_sem_registros       EXCEPTION;      -- JOIN nao retornou linhas
    e_falha_conversao     EXCEPTION;      -- Funcao 1 devolveu NULL para alguma linha
    v_nome_clinica  CLINICA.nome%TYPE;
    v_json          CLOB;                 -- array JSON completo
    v_obj_pet       VARCHAR2(4000);       -- objeto aninhado "pet"
    v_obj_tutor     VARCHAR2(4000);       -- objeto aninhado "tutor"
    v_obj           VARCHAR2(32767);      -- objeto do atendimento
    v_qtd           PLS_INTEGER := 0;
    v_id_erro       ATENDIMENTO.id_atendimento%TYPE;
    -- JOIN da tabela fato com suas 5 dimensoes
    CURSOR c_atend IS
        SELECT a.id_atendimento, a.data_atendimento, a.valor, a.status,
               c.nome            AS clinica,
               c.cidade,
               v.nome            AS veterinario,
               v.crmv,
               v.especialidade,
               s.descricao       AS servico,
               p.id_pet, p.nome  AS pet, p.especie, p.raca, p.sexo, p.peso_kg,
               t.id_tutor, t.nome AS tutor, t.telefone, t.email
          FROM ATENDIMENTO   a
          JOIN CLINICA       c ON c.id_clinica      = a.id_clinica
          JOIN VETERINARIO   v ON v.id_veterinario  = a.id_veterinario
          JOIN TIPO_SERVICO  s ON s.id_tipo_servico = a.id_tipo_servico
          JOIN PET           p ON p.id_pet          = a.id_pet
          JOIN TUTOR         t ON t.id_tutor        = p.id_tutor
         WHERE (p_clinica_id IS NULL OR a.id_clinica = p_clinica_id)
         ORDER BY a.id_clinica, a.data_atendimento, a.id_atendimento;
BEGIN
    -- 1) validacao do parametro
    IF p_clinica_id IS NOT NULL AND p_clinica_id <= 0 THEN
        RAISE e_parametro_invalido;
    END IF;
    -- 2) clinica informada precisa existir (NO_DATA_FOUND caso contrario)
    IF p_clinica_id IS NOT NULL THEN
        SELECT nome INTO v_nome_clinica FROM CLINICA WHERE id_clinica = p_clinica_id;
    END IF;
    DBMS_OUTPUT.PUT_LINE('== Atendimentos em JSON'
                         || CASE WHEN p_clinica_id IS NOT NULL
                                 THEN ' - clinica ' || p_clinica_id || ' (' || v_nome_clinica || ')'
                                 ELSE ' - todas as clinicas' END || ' ==');
    -- 3) uma linha do JOIN => um objeto JSON; objetos separados por virgula dentro do array
    v_json := '[' || CHR(10);
    FOR r IN c_atend LOOP
        v_qtd     := v_qtd + 1;
        v_id_erro := r.id_atendimento;
        -- objeto aninhado: pet
```

```
        v_obj_pet := FN_MONTA_JSON(
            'id|nome|especie|raca|sexo|peso_kg',
            r.id_pet || '|' || r.pet || '|' || r.especie || '|' || r.raca || '|' || r.sexo || '|' ||
        TO_CHAR(r.peso_kg),
            'N|S|S|S|S|N');
        -- objeto aninhado: tutor
        v_obj_tutor := FN_MONTA_JSON(
            'id|nome|telefone|email',
            r.id_tutor || '|' || r.tutor || '|' || r.telefone || '|' || r.email,
            'N|S|S|S');
        -- objeto principal: atendimento (com pet e tutor embutidos via tipo J)
        v_obj := FN_MONTA_JSON(
            'id_atendimento|data|clinica|cidade|veterinario|crmv|especialidade|servico|valor|realizado|pet|tutor',
            r.id_atendimento                          || '|' ||
            TO_CHAR(r.data_atendimento, 'DD/MM/YYYY')  || '|' ||
            r.clinica                                 || '|' ||
            r.cidade                                  || '|' ||
            r.veterinario                             || '|' ||
            r.crmv                                    || '|' ||
            r.especialidade                           || '|' ||
            r.servico                                 || '|' ||
            TO_CHAR(r.valor, 'FM9999999990.00')       || '|' ||
            CASE WHEN r.status = 'REALIZADO' THEN 'S' ELSE 'N' END || '|' ||
            v_obj_pet                                 || '|' ||
            v_obj_tutor,
            'N|S|S|S|S|S|S|S|N|B|J|J');
        IF v_obj IS NULL OR v_obj_pet IS NULL OR v_obj_tutor IS NULL THEN
            RAISE e_falha_conversao;
        END IF;
        IF v_qtd > 1 THEN
            v_json := v_json || ',' || CHR(10);
        END IF;
        v_json := v_json || '  ' || v_obj;
    END LOOP;
    IF v_qtd = 0 THEN
        RAISE e_sem_registros;
    END IF;
    v_json := v_json || CHR(10) || ']';
    -- 4) saida
    PRC_IMPRIME_CLOB(v_json);
    DBMS_OUTPUT.PUT_LINE('== ' || v_qtd || ' atendimento(s) convertido(s) | ' || DBMS_LOB.GETLENGTH(v_json) || '
        caracteres ==');
EXCEPTION
    WHEN e_parametro_invalido THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_ATENDIMENTOS_JSON] Erro tratado (e_parametro_invalido): o id da clinica deve ser
        positivo. Recebido: '
                             || p_clinica_id);
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_ATENDIMENTOS_JSON] Erro tratado (NO_DATA_FOUND): nao existe clinica com id ' ||
        p_clinica_id || '.');
    WHEN e_sem_registros THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_ATENDIMENTOS_JSON] Erro tratado (e_sem_registros): a clinica ' || p_clinica_id || '
         ('
                             || v_nome_clinica || ') ainda nao possui atendimentos para converter.');
    WHEN e_falha_conversao THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_ATENDIMENTOS_JSON] Erro tratado (e_falha_conversao): FN_MONTA_JSON nao conseguiu
        converter o atendimento '
                             || v_id_erro || '. Veja a mensagem da funcao acima.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_ATENDIMENTOS_JSON] Erro inesperado ' || SQLCODE || ': ' || SQLERRM);
END PRC_ATENDIMENTOS_JSON;
/
SHOW ERRORS
```

7.3 Execução

Bloco 10.6: os 8 atendimentos da clínica 1 (ClyvoVet Paulista), incluindo o histórico de 2025. Bloco 10.6b: as seis clínicas — 34 objetos, 15 mil caracteres, impressos pelo utilitário sem truncamento.

```
-- 10.6 PROCEDURE 1 - execucao normal: JOIN -> JSON (uma clinica e todas)
```

Saida do script | 10.6 PRC_ATENDIMENTOS_JSON - execucao normal

```
== Atendimentos em JSON - clinica 1 (ClyvoVet Paulista) ==
[
  {"id_atendimento":31,"data":"15/11/2025","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta clinica","valor":170
      .00,"realizado":true,"pet":{"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","peso_kg":32.5},"
      tutor":{"id":1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":32,"data":"20/12/2025","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Vacinacao","valor":110.00,"re
      alizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}},
  {"id_atendimento":1,"data":"10/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta clinica","valor":180
      .00,"realizado":true,"pet":{"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","peso_kg":32.5},"
      tutor":{"id":1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":2,"data":"12/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Vacinacao","valor":120.00,"re
      alizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}},
  {"id_atendimento":3,"data":"15/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Juliana Ferreira","crmv":"CRMV-SP 34567","especialidade":"Dermatologia","servico":"Exames laboratoriais","valo
      r":250.00,"realizado":true,"pet":{"id":3,"nome":"Luna","especie":"GATO","raca":"Siames","sexo":"F","peso_kg":4
      .2},"tutor":{"id":3,"nome":"Carla Mendes Souza","telefone":"(11)
      98888-1003","email":"carla.souza@email.com"}},
  {"id_atendimento":4,"data":"20/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":1500.00,"realiz
      ado":true,"pet":{"id":4,"nome":"Thor","especie":"CAO","raca":"Pastor
      Alemao","sexo":"M","peso_kg":38},"tutor":{"id":4,"nome":"Diego Fernandes Alves","telefone":"(11)
      98888-1004","email":"diego.alves@email.com"}},
  {"id_atendimento":5,"data":"22/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Patricia Gomes","crmv":"CRMV-SP 56789","especialidade":"Diagnostico por imagem","servico":"Ultrassonografia","
      valor":320.00,"realizado":true,"pet":{"id":5,"nome":"Nina","especie":"GATO","raca":"Persa","sexo":"F","peso_kg
      ":3.8},"tutor":{"id":5,"nome":"Elaine Rocha Martins","telefone":"(11)
      98888-1005","email":"elaine.martins@email.com"}},
  {"id_atendimento":6,"data":"05/02/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta
      clinica","valor":195.00,"realizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}}
]
== 8 atendimento(s) convertido(s) | 3539 caracteres ==

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 7 — PRC_ATENDIMENTOS_JSON(1): array JSON com objetos pet e tutor aninhados.

Saida do script | 10.6b PRC_ATENDIMENTOS_JSON - todas as clinicas (parte 1/3)

```
== Atendimentos em JSON - todas as clinicas ==
[
  {"id_atendimento":31,"data":"15/11/2025","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta clinica","valor":170
      .00,"realizado":true,"pet":{"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","peso_kg":32.5},"
      tutor":{"id":1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":32,"data":"20/12/2025","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Vacinacao","valor":110.00,"re
      alizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}},
  {"id_atendimento":1,"data":"10/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta clinica","valor":180
      .00,"realizado":true,"pet":{"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","peso_kg":32.5},"
      tutor":{"id":1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":2,"data":"12/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Vacinacao","valor":120.00,"re
      alizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}},
  {"id_atendimento":3,"data":"15/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Juliana Ferreira","crmv":"CRMV-SP 34567","especialidade":"Dermatologia","servico":"Exames laboratoriais","valo
      r":250.00,"realizado":true,"pet":{"id":3,"nome":"Luna","especie":"GATO","raca":"Siames","sexo":"F","peso_kg":4
      .2},"tutor":{"id":3,"nome":"Carla Mendes Souza","telefone":"(11)
      98888-1003","email":"carla.souza@email.com"}},
  {"id_atendimento":4,"data":"20/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":1500.00,"realiz
      ado":true,"pet":{"id":4,"nome":"Thor","especie":"CAO","raca":"Pastor
      Alemao","sexo":"M","peso_kg":38},"tutor":{"id":4,"nome":"Diego Fernandes Alves","telefone":"(11)
      98888-1004","email":"diego.alves@email.com"}},
  {"id_atendimento":5,"data":"22/01/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Patricia Gomes","crmv":"CRMV-SP 56789","especialidade":"Diagnostico por imagem","servico":"Ultrassonografia","
      valor":320.00,"realizado":true,"pet":{"id":5,"nome":"Nina","especie":"GATO","raca":"Persa","sexo":"F","peso_kg
      ":3.8},"tutor":{"id":5,"nome":"Elaine Rocha Martins","telefone":"(11)
      98888-1005","email":"elaine.martins@email.com"}},
  {"id_atendimento":6,"data":"05/02/2026","clinica":"ClyvoVet Paulista","cidade":"Sao Paulo","veterinario":"Dra.
      Mariana Costa","crmv":"CRMV-SP 12345","especialidade":"Clinica geral","servico":"Consulta
      clinica","valor":195.00,"realizado":true,"pet":{"id":2,"nome":"Mel","especie":"CAO","raca":"Golden
      Retriever","sexo":"F","peso_kg":28},"tutor":{"id":2,"nome":"Bruno Carvalho Lima","telefone":"(11)
      98888-1002","email":"bruno.lima@email.com"}},
  {"id_atendimento":7,"data":"03/02/2026","clinica":"ClyvoVet Pinheiros","cidade":"Sao Paulo","veterinario":"Dr.
      Lucas Moreira","crmv":"CRMV-SP 67890","especialidade":"Clinica geral","servico":"Consulta clinica","valor":180
      .00,"realizado":true,"pet":{"id":6,"nome":"Bob","especie":"CAO","raca":"Beagle","sexo":"M","peso_kg":14.5},"tu
      tor":{"id":6,"nome":"Fabio Nogueira Prado","telefone":"(11) 98888-1006","email":"fabio.prado@email.com"}},
  {"id_atendimento":8,"data":"04/02/2026","clinica":"ClyvoVet Pinheiros","cidade":"Sao Paulo","veterinario":"Dr.
      Lucas Moreira","crmv":"CRMV-SP 67890","especialidade":"Clinica geral","servico":"Vacinacao","valor":135.00,"re
      alizado":true,"pet":{"id":7,"nome":"Mia","especie":"GATO","raca":"SRD","sexo":"F","peso_kg":4.5},"tutor":{"id"
      :7,"nome":"Gabriela Santos Teixeira","telefone":"(11) 98888-1007","email":"gabriela.teixeira@email.com"}},
  {"id_atendimento":9,"data":"10/02/2026","clinica":"ClyvoVet Pinheiros","cidade":"Sao Paulo","veterinario":"Dra.
      Juliana Ferreira","crmv":"CRMV-SP 34567","especialidade":"Dermatologia","servico":"Exames
      laboratoriais","valor":275.00,"realizado":true,"pet":{"id":8,"nome":"Simba","especie":"GATO","raca":"Maine
      Coon","sexo":"M","peso_kg":7.8},"tutor":{"id":8,"nome":"Henrique Duarte Campos","telefone":"(11)
      98888-1008","email":"henrique.campos@email.com"}},
  {"id_atendimento":10,"data":"14/02/2026","clinica":"ClyvoVet Pinheiros","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":1800.00,"realiz
      ado":true,"pet":{"id":9,"nome":"Lola","especie":"CAO","raca":"Poodle","sexo":"F","peso_kg":6.2},"tutor":{"id":
      1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":11,"data":"18/02/2026","clinica":"ClyvoVet Pinheiros","cidade":"Sao Paulo","veterinario":"Dra.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 8 — PRC_ATENDIMENTOS_JSON sem filtro: início e fim da saída com as 34 conversões. (parte 1 de 3)

Saida do script | 10.6b PRC_ATENDIMENTOS_JSON - todas as clinicas (parte 3/3)


```
      "tutor":{"id":6,"nome":"Fabio Nogueira Prado","telefone":"(11) 98888-1006","email":"fabio.prado@email.com"}},
  {"id_atendimento":22,"data":"10/04/2026","clinica":"ClyvoVet Santana","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":2100.00,"realiz
      ado":true,"pet":{"id":8,"nome":"Simba","especie":"GATO","raca":"Maine
      Coon","sexo":"M","peso_kg":7.8},"tutor":{"id":8,"nome":"Henrique Duarte Campos","telefone":"(11)
      98888-1008","email":"henrique.campos@email.com"}},
  {"id_atendimento":23,"data":"14/04/2026","clinica":"ClyvoVet Santana","cidade":"Sao Paulo","veterinario":"Dra.
      Patricia Gomes","crmv":"CRMV-SP 56789","especialidade":"Diagnostico por imagem","servico":"Ultrassonografia","
      valor":320.00,"realizado":true,"pet":{"id":10,"nome":"Zeus","especie":"CAO","raca":"Rottweiler","sexo":"M","pe
      so_kg":45},"tutor":{"id":4,"nome":"Diego Fernandes Alves","telefone":"(11)
      98888-1004","email":"diego.alves@email.com"}},
  {"id_atendimento":24,"data":"21/04/2026","clinica":"ClyvoVet Santana","cidade":"Sao Paulo","veterinario":"Dr.
      Thiago Barbosa","crmv":"CRMV-SP 45678","especialidade":"Cardiologia","servico":"Exames
      laboratoriais","valor":250.00,"realizado":true,"pet":{"id":4,"nome":"Thor","especie":"CAO","raca":"Pastor
      Alemao","sexo":"M","peso_kg":38},"tutor":{"id":4,"nome":"Diego Fernandes Alves","telefone":"(11)
      98888-1004","email":"diego.alves@email.com"}},
  {"id_atendimento":25,"data":"04/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dr.
      Lucas Moreira","crmv":"CRMV-SP 67890","especialidade":"Clinica geral","servico":"Consulta clinica","valor":180
      .00,"realizado":true,"pet":{"id":3,"nome":"Luna","especie":"GATO","raca":"Siames","sexo":"F","peso_kg":4.2},"t
      utor":{"id":3,"nome":"Carla Mendes Souza","telefone":"(11) 98888-1003","email":"carla.souza@email.com"}},
  {"id_atendimento":26,"data":"06/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dr.
      Lucas Moreira","crmv":"CRMV-SP 67890","especialidade":"Clinica geral","servico":"Vacinacao","valor":120.00,"re
      alizado":true,"pet":{"id":5,"nome":"Nina","especie":"GATO","raca":"Persa","sexo":"F","peso_kg":3.8},"tutor":{"
      id":5,"nome":"Elaine Rocha Martins","telefone":"(11) 98888-1005","email":"elaine.martins@email.com"}},
  {"id_atendimento":27,"data":"11/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dra.
      Juliana Ferreira","crmv":"CRMV-SP 34567","especialidade":"Dermatologia","servico":"Exames laboratoriais","valo
      r":250.00,"realizado":true,"pet":{"id":7,"nome":"Mia","especie":"GATO","raca":"SRD","sexo":"F","peso_kg":4.5},
      "tutor":{"id":7,"nome":"Gabriela Santos Teixeira","telefone":"(11)
      98888-1007","email":"gabriela.teixeira@email.com"}},
  {"id_atendimento":28,"data":"15/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":1500.00,"realiz
      ado":true,"pet":{"id":9,"nome":"Lola","especie":"CAO","raca":"Poodle","sexo":"F","peso_kg":6.2},"tutor":{"id":
      1,"nome":"Ana Paula Ribeiro","telefone":"(11) 98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":29,"data":"19/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dra.
      Patricia Gomes","crmv":"CRMV-SP 56789","especialidade":"Diagnostico por imagem","servico":"Ultrassonografia","
      valor":320.00,"realizado":true,"pet":{"id":1,"nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","peso_k
      g":32.5},"tutor":{"id":1,"nome":"Ana Paula Ribeiro","telefone":"(11)
      98888-1001","email":"ana.ribeiro@email.com"}},
  {"id_atendimento":30,"data":"26/05/2026","clinica":"ClyvoVet Tatuape","cidade":"Sao Paulo","veterinario":"Dr.
      Rafael Almeida","crmv":"CRMV-SP 23456","especialidade":"Cirurgia","servico":"Cirurgia","valor":1750.00,"realiz
      ado":true,"pet":{"id":10,"nome":"Zeus","especie":"CAO","raca":"Rottweiler","sexo":"M","peso_kg":45},"tutor":{"
      id":4,"nome":"Diego Fernandes Alves","telefone":"(11) 98888-1004","email":"diego.alves@email.com"}}
]
== 34 atendimento(s) convertido(s) | 15042 caracteres ==

PL/SQL procedure successfully completed.
```


Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 8 — PRC_ATENDIMENTOS_JSON sem filtro: início e fim da saída com as 34 conversões. (parte 3 de 3)

7.4 Exceções tratadas

```
-- 10.7 PROCEDURE 1 - excecoes tratadas
```

Saida do script | 10.7 PRC_ATENDIMENTOS_JSON - excecoes tratadas

```
[PRC_ATENDIMENTOS_JSON] Erro tratado (NO_DATA_FOUND): nao existe clinica com id 999.
== Atendimentos em JSON - clinica 6 (ClyvoVet Campinas) ==
[PRC_ATENDIMENTOS_JSON] Erro tratado (e_sem_registros): a clinica 6 (ClyvoVet Campinas) ainda nao possui
      atendimentos para converter.
[PRC_ATENDIMENTOS_JSON] Erro tratado (e_parametro_invalido): o id da clinica deve ser positivo. Recebido: -1

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 9 — PRC_ATENDIMENTOS_JSON: NO_DATA_FOUND (clínica 999), e_sem_registros (clínica 6) e e_parametro_invalido (-1).

8. Procedimento 2 — PRC_FATURAMENTO_CLINICA

8.1 Finalidade e técnica

Lê a tabela fato ATENDIMENTO (colunas categóricas *id_clinica* e *id_tipo_servico*, medida *valor*) e exibe o faturamento por combinação, o subtotal por clínica e o total geral, no formato do enunciado. O somatório é inteiramente manual, por **quebra de controle**: um cursor ordenado pela hierarquia das categorias é percorrido uma única vez enquanto três acumuladores são atualizados. Não há SUM, GROUP BY, ROLLUP, CUBE, GROUPING SETS ou GROUPING em nenhum ponto. Só entram atendimentos com status REALIZADO; o ano chega como texto de propósito, para que a conversão seja validada manualmente (VALUE_ERROR se não numérico, e_ano_invalido fora de 2000–2100).

8.2 Código

```
-- ================================================================================
-- [8] PROCEDURE 2 - PRC_FATURAMENTO_CLINICA
--     Le a tabela fato ATENDIMENTO e exibe o faturamento por (clinica, servico), o
--     subtotal por clinica e o total geral, no formato exigido pelo enunciado.
--
--     Tecnica: QUEBRA DE CONTROLE (control break) em uma unica passagem por um cursor
--     ordenado pela hierarquia das categorias. Tres acumuladores PL/SQL fazem todo o
--     somatorio: v_soma_combo (clinica+servico), v_soma_clinica (subtotal) e v_total_geral.
--     Nao ha SUM, GROUP BY, ROLLUP, CUBE, GROUPING SETS ou GROUPING em lugar nenhum.
--     Nas linhas de subtotal e total geral as colunas de agrupamento ficam em branco (nulas).
--
--     p_ano: recebido como texto de proposito, para a conversao ser validada manualmente.
--     Excecoes tratadas: VALUE_ERROR/INVALID_NUMBER, e_ano_invalido, e_sem_movimento, OTHERS.
-- ================================================================================
CREATE OR REPLACE PROCEDURE PRC_FATURAMENTO_CLINICA (p_ano IN VARCHAR2)
IS
    e_ano_invalido    EXCEPTION;          -- ano fora da faixa 2000..2100
    e_sem_movimento   EXCEPTION;          -- nenhum atendimento realizado no ano
    v_ano             NUMBER;
    v_clinica_atual   ATENDIMENTO.id_clinica%TYPE;
    v_servico_atual   ATENDIMENTO.id_tipo_servico%TYPE;
    v_soma_combo      NUMBER := 0;        -- acumulador da combinacao (clinica, servico)
    v_soma_clinica    NUMBER := 0;        -- acumulador do subtotal da clinica
    v_total_geral     NUMBER := 0;        -- acumulador do total geral
    v_qtd_linhas      PLS_INTEGER := 0;
    v_primeira        BOOLEAN := TRUE;
    -- A ordenacao pela hierarquia (clinica -> servico) e o que permite a quebra de controle.
    CURSOR c_fato IS
        SELECT id_clinica, id_tipo_servico, valor
          FROM ATENDIMENTO
         WHERE status = 'REALIZADO'
           AND EXTRACT(YEAR FROM data_atendimento) = v_ano
         ORDER BY id_clinica, id_tipo_servico;
    -- valor com 2 casas e ponto decimal, alinhado a direita em 10 posicoes
    FUNCTION fmt_valor(p_valor NUMBER) RETURN VARCHAR2 IS
    BEGIN
        RETURN LPAD(TO_CHAR(p_valor, 'FM9999999990.00', 'NLS_NUMERIC_CHARACTERS=''.,'''), 10);
    END fmt_valor;
    PROCEDURE linha_detalhe IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE(LPAD(v_clinica_atual, 7) || '  ' || LPAD(v_servico_atual, 7) || '  ' ||
        fmt_valor(v_soma_combo));
    END linha_detalhe;
    -- colunas Clinica e Servico ficam vazias; o rotulo ocupa a area das duas colunas
    PROCEDURE linha_subtotal IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE(RPAD('  Sub Total', 16) || '  ' || fmt_valor(v_soma_clinica));
    END linha_subtotal;
BEGIN
    -- 1) validacao manual do parametro (VALUE_ERROR se nao for numerico)
    v_ano := TO_NUMBER(p_ano);
    IF v_ano IS NULL OR v_ano <> TRUNC(v_ano) OR v_ano < 2000 OR v_ano > 2100 THEN
        RAISE e_ano_invalido;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Faturamento por clinica e servico - atendimentos REALIZADOS em ' || v_ano);
    DBMS_OUTPUT.PUT_LINE(RPAD('Clinica', 7) || '  ' || RPAD('Servico', 7) || '  ' || LPAD('Valor', 10));
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 7, '-')  || '  ' || RPAD('-', 7, '-')  || '  ' || RPAD('-', 10, '-'));
    -- 2) quebra de controle: uma unica passagem pelo cursor ordenado
```

```
    FOR r IN c_fato LOOP
        v_qtd_linhas := v_qtd_linhas + 1;
        IF v_primeira THEN
            v_clinica_atual := r.id_clinica;
            v_servico_atual := r.id_tipo_servico;
            v_primeira      := FALSE;
        END IF;
        -- mudou a combinacao (clinica, servico)? fecha a linha de detalhe acumulada
        IF r.id_clinica <> v_clinica_atual OR r.id_tipo_servico <> v_servico_atual THEN
            linha_detalhe;
            v_soma_combo := 0;
            -- mudou a clinica? fecha tambem o subtotal
            IF r.id_clinica <> v_clinica_atual THEN
                linha_subtotal;
                v_soma_clinica := 0;
            END IF;
            v_clinica_atual := r.id_clinica;
            v_servico_atual := r.id_tipo_servico;
        END IF;
        -- somatorio manual nos tres niveis
        v_soma_combo   := v_soma_combo   + r.valor;
        v_soma_clinica := v_soma_clinica + r.valor;
        v_total_geral  := v_total_geral  + r.valor;
    END LOOP;
    IF v_qtd_linhas = 0 THEN
        RAISE e_sem_movimento;
    END IF;
    -- 3) fecha o ultimo grupo pendente e imprime o total geral
    linha_detalhe;
    linha_subtotal;
    DBMS_OUTPUT.PUT_LINE(RPAD('Total Geral', 16) || '  ' || fmt_valor(v_total_geral));
    DBMS_OUTPUT.PUT_LINE('(' || v_qtd_linhas || ' atendimento(s) somado(s))');
EXCEPTION
    WHEN VALUE_ERROR OR INVALID_NUMBER THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_FATURAMENTO_CLINICA] Erro tratado (VALUE_ERROR): o ano deve ser numerico. Recebido:
         "' || p_ano || '".');
    WHEN e_ano_invalido THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_FATURAMENTO_CLINICA] Erro tratado (e_ano_invalido): ano fora da faixa aceita (2000
        a 2100). Recebido: "'
                             || p_ano || '".');
    WHEN e_sem_movimento THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_FATURAMENTO_CLINICA] Erro tratado (e_sem_movimento): nenhum atendimento REALIZADO
        em ' || v_ano || '.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_FATURAMENTO_CLINICA] Erro inesperado ' || SQLCODE || ': ' || SQLERRM);
END PRC_FATURAMENTO_CLINICA;
/
SHOW ERRORS
```

8.3 Execução

Bloco 10.8: ano 2026 — 30 atendimentos, 25 combinações, cinco subtotais (2 565,00 · 2 830,00 · 2 890,00 · 3 230,00 · 4 120,00) e total geral 15 635,00. Os dois atendimentos AGENDADOS e os dois de 2025 ficam de fora. Bloco 10.8b: ano 2025, com apenas duas combinações da clínica 1.

```
-- 10.8 PROCEDURE 2 - execucao normal: subtotais por clinica e total geral
```

Saida do script | 10.8 PRC_FATURAMENTO_CLINICA - execucao normal

```
Faturamento por clinica e servico - atendimentos REALIZADOS em 2026
Clinica  Servico       Valor
-------  -------  ----------
      1        1      375.00
      1        2      120.00
      1        3      250.00
      1        4     1500.00
      1        5      320.00
  Sub Total          2565.00
      2        1      180.00
      2        2      255.00
      2        3      275.00
      2        4     1800.00
      2        5      320.00
  Sub Total          2830.00
      3        1      200.00
      3        2      120.00
      3        3      250.00
      3        4     1650.00
      3        5      670.00
  Sub Total          2890.00
      4        1      180.00
      4        2      120.00
      4        3      510.00
      4        4     2100.00
      4        5      320.00
  Sub Total          3230.00
      5        1      180.00
      5        2      120.00
      5        3      250.00
      5        4     3250.00
      5        5      320.00
  Sub Total          4120.00
Total Geral         15635.00
(30 atendimento(s) somado(s))

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 10 — PRC_FATURAMENTO_CLINICA('2026'): detalhe, subtotais por clínica e total geral.

Saida do script | 10.8b PRC_FATURAMENTO_CLINICA - outro ano (2025)

```
Faturamento por clinica e servico - atendimentos REALIZADOS em 2025
Clinica  Servico       Valor
-------  -------  ----------
      1        1      170.00
      1        2      110.00
  Sub Total           280.00
Total Geral           280.00
(2 atendimento(s) somado(s))

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 11 — PRC_FATURAMENTO_CLINICA('2025').

8.4 Exceções tratadas

```
-- 10.9 PROCEDURE 2 - excecoes tratadas
```

Saida do script | 10.9 PRC_FATURAMENTO_CLINICA - excecoes tratadas

```
[PRC_FATURAMENTO_CLINICA] Erro tratado (VALUE_ERROR): o ano deve ser numerico. Recebido: "ABC".
[PRC_FATURAMENTO_CLINICA] Erro tratado (e_ano_invalido): ano fora da faixa aceita (2000 a 2100). Recebido: "1990".
Faturamento por clinica e servico - atendimentos REALIZADOS em 2031
Clinica  Servico       Valor
-------  -------  ----------
[PRC_FATURAMENTO_CLINICA] Erro tratado (e_sem_movimento): nenhum atendimento REALIZADO em 2031.

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 12 — PRC_FATURAMENTO_CLINICA: VALUE_ERROR ('ABC'), e_ano_invalido ('1990') e e_sem_movimento ('2031').

9. Trigger de auditoria — TRG_AUD_PET

9.1 Tabela de auditoria

AUD_PET guarda, para cada operação em PET: o usuário Oracle (USER), o usuário do sistema operacional e a máquina de origem (SYS_CONTEXT), o tipo da operação (INSERT, UPDATE, DELETE), a data e hora (SYSTIMESTAMP), o identificador do registro afetado e os valores anteriores (:OLD) e novos (:NEW) serializados em JSON pela Função 1. No INSERT o valor anterior é nulo; no DELETE, o valor novo.

```
-- Tabela de AUDITORIA exigida pelo trabalho: usuario, operacao, data/hora,
-- valores anteriores (:OLD) e valores novos (:NEW). OLD/NEW sao gravados como JSON
-- gerado pela Funcao 1, o que reaproveita a conversao manual em mais um ponto do projeto.
CREATE TABLE AUD_PET (
    id_auditoria    NUMBER(10)      CONSTRAINT pk_aud_pet PRIMARY KEY,
    nome_tabela     VARCHAR2(30)    NOT NULL,
    id_registro     NUMBER(6),                  -- chave do registro afetado em PET
    tipo_operacao   VARCHAR2(10)    NOT NULL
                    CONSTRAINT ck_aud_pet_operacao CHECK (tipo_operacao IN ('INSERT', 'UPDATE', 'DELETE')),
    nome_usuario    VARCHAR2(100)   NOT NULL,   -- usuario Oracle (USER)
    usuario_so      VARCHAR2(100),              -- usuario do sistema operacional do cliente
    maquina         VARCHAR2(100),              -- host de origem da sessao
    data_hora       TIMESTAMP       NOT NULL,
    valor_anterior  CLOB,                       -- JSON com :OLD (nulo no INSERT)
    valor_novo      CLOB                        -- JSON com :NEW (nulo no DELETE)
);
```

9.2 Código da trigger

```
-- ================================================================================
-- [9] AUDITORIA - TRG_AUD_PET
--     AFTER INSERT OR UPDATE OR DELETE ON PET, por linha. Grava em AUD_PET o usuario,
--     a operacao, a data/hora e os valores anteriores (:OLD) e novos (:NEW) - estes dois
--     serializados em JSON pela Funcao 1.
--
--     Por que nao ha risco de "mutating table" (ORA-04091): FN_MONTA_JSON recebe apenas
--     os valores escalares de :OLD/:NEW e nao consulta a tabela PET.
--     A trigger e criada DEPOIS da carga inicial para que a auditoria contenha apenas
--     as operacoes do roteiro de testes.
-- ================================================================================
CREATE OR REPLACE TRIGGER TRG_AUD_PET
AFTER INSERT OR UPDATE OR DELETE ON PET
FOR EACH ROW
DECLARE
    c_chaves     CONSTANT VARCHAR2(200) := 'id_pet|id_tutor|nome|especie|raca|sexo|data_nascimento|peso_kg';
    c_tipos      CONSTANT VARCHAR2(50)  := 'N|N|S|S|S|S|S|N';
    v_operacao   AUD_PET.tipo_operacao%TYPE;
    v_anterior   CLOB;
    v_novo       CLOB;
BEGIN
    -- identifica a operacao
    IF INSERTING THEN
        v_operacao := 'INSERT';
    ELSIF UPDATING THEN
        v_operacao := 'UPDATE';
    ELSE
        v_operacao := 'DELETE';
    END IF;
    -- valores anteriores (:OLD) - nao existem no INSERT
    IF UPDATING OR DELETING THEN
        v_anterior := FN_MONTA_JSON(c_chaves,
            :OLD.id_pet || '|' || :OLD.id_tutor || '|' || :OLD.nome || '|' || :OLD.especie || '|' || :OLD.raca || '|'
        ||
            :OLD.sexo || '|' || TO_CHAR(:OLD.data_nascimento, 'DD/MM/YYYY') || '|' || TO_CHAR(:OLD.peso_kg),
            c_tipos);
    END IF;
    -- valores novos (:NEW) - nao existem no DELETE
    IF INSERTING OR UPDATING THEN
        v_novo := FN_MONTA_JSON(c_chaves,
            :NEW.id_pet || '|' || :NEW.id_tutor || '|' || :NEW.nome || '|' || :NEW.especie || '|' || :NEW.raca || '|'
        ||
            :NEW.sexo || '|' || TO_CHAR(:NEW.data_nascimento, 'DD/MM/YYYY') || '|' || TO_CHAR(:NEW.peso_kg),
```

```
            c_tipos);
    END IF;
    INSERT INTO AUD_PET (id_auditoria, nome_tabela, id_registro, tipo_operacao,
                         nome_usuario, usuario_so, maquina, data_hora, valor_anterior, valor_novo)
    VALUES (SEQ_AUD_PET.NEXTVAL, 'PET', NVL(:NEW.id_pet, :OLD.id_pet), v_operacao,
            USER, SYS_CONTEXT('USERENV', 'OS_USER'), SYS_CONTEXT('USERENV', 'HOST'),
            SYSTIMESTAMP, v_anterior, v_novo);
EXCEPTION
    WHEN OTHERS THEN
        -- Auditoria e obrigatoria: o erro nao e engolido. A mensagem identifica a origem
        -- e a operacao em PET e desfeita junto com a falha da trigger.
        RAISE_APPLICATION_ERROR(-20051, 'TRG_AUD_PET: falha ao registrar auditoria da operacao '
                                        || v_operacao || ' em PET - ' || SQLERRM);
END TRG_AUD_PET;
/
SHOW ERRORS
```

9.3 Execução

Bloco 10.10: dois INSERTs (pets Bidu e Frajola, criados pela SEQ_PET), dois UPDATEs (peso do Bidu; nome e raça do Frajola) e dois DELETEs — seis operações, seis linhas de auditoria, e a tabela PET volta aos dez registros originais. Bloco 10.11: consulta à AUD_PET; bloco 10.11b: a mesma trilha em formato de relatório, com o JSON completo de :OLD e :NEW lado a lado.

```
-- 10.10 TRIGGER - operacoes DML em PET (2 INSERT, 2 UPDATE, 2 DELETE = 6 auditorias)
--       Os pets de teste sao criados e removidos aqui mesmo, sem registros filhos.
```

Saida do script | 10.10 TRG_AUD_PET - operacoes DML auditadas

```
1 row created.

1 row created.

1 row updated.

1 row updated.

1 row deleted.

1 row deleted.

Commit complete.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 13 — Operações DML em PET disparando a trigger.

Saida do script | 10.11 AUD_PET - registros gravados pela trigger

```
ID_AUDITORIA TIPO_OPER ID_REGISTRO NOME_USUARIO USUARIO_SO   MAQUINA          DATA_HORA           TEM_OLD TEM_NEW
------------ --------- ----------- ------------ ------------ ---------------- ------------------- ------- -------
           1 INSERT             11 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 -       sim
           2 INSERT             12 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 -       sim
           3 UPDATE             11 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 sim     sim
           4 UPDATE             12 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 sim     sim
           5 DELETE             12 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 sim     -
           6 DELETE             11 CLYVO        oracle       24e319fb9c4f     12/09/2026 16:30:39 sim     -

6 rows selected.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 14 — SELECT na tabela AUD_PET: quem, o que e quando (usuário Oracle, usuário do SO, máquina, data/hora).

Saida do script | 10.11b AUD_PET - trilha de auditoria legivel

```
#1 INSERT | PET id 11 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: (nulo - registro nao existia)
   novo    : {"id_pet":11,"id_tutor":2,"nome":"Bidu","especie":"CAO","raca":"SRD","sexo":"M","data_nascimento":"01/0
     1/2024","peso_kg":9.8}
#2 INSERT | PET id 12 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: (nulo - registro nao existia)
   novo    : {"id_pet":12,"id_tutor":3,"nome":"Frajola","especie":"GATO","raca":"SRD","sexo":"M","data_nascimento":"
     15/05/2023","peso_kg":4.1}
#3 UPDATE | PET id 11 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: {"id_pet":11,"id_tutor":2,"nome":"Bidu","especie":"CAO","raca":"SRD","sexo":"M","data_nascimento":"01/0
     1/2024","peso_kg":9.8}
   novo    : {"id_pet":11,"id_tutor":2,"nome":"Bidu","especie":"CAO","raca":"SRD","sexo":"M","data_nascimento":"01/0
     1/2024","peso_kg":10.4}
#4 UPDATE | PET id 12 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: {"id_pet":12,"id_tutor":3,"nome":"Frajola","especie":"GATO","raca":"SRD","sexo":"M","data_nascimento":"
     15/05/2023","peso_kg":4.1}
   novo    : {"id_pet":12,"id_tutor":3,"nome":"Frajola
     Jr.","especie":"GATO","raca":"Bombaim","sexo":"M","data_nascimento":"15/05/2023","peso_kg":4.1}
#5 DELETE | PET id 12 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: {"id_pet":12,"id_tutor":3,"nome":"Frajola
     Jr.","especie":"GATO","raca":"Bombaim","sexo":"M","data_nascimento":"15/05/2023","peso_kg":4.1}
   novo    : (nulo - registro removido)
#6 DELETE | PET id 11 | usuario CLYVO (oracle@24e319fb9c4f) | 12/09/2026 16:30:39
   anterior: {"id_pet":11,"id_tutor":2,"nome":"Bidu","especie":"CAO","raca":"SRD","sexo":"M","data_nascimento":"01/0
     1/2024","peso_kg":10.4}
   novo    : (nulo - registro removido)

PL/SQL procedure successfully completed.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 15 — Trilha de auditoria completa: valores anteriores (:OLD) e novos (:NEW) em JSON para cada operação.

10. Conferências finais

A execução do script inteiro termina sem nenhum erro de compilação ou execução: todos os objetos ficam com status VALID e todas as tabelas — inclusive a de auditoria — têm pelo menos cinco registros.

Saida do script | Compilacao dos objetos (SQL*Plus) (parte 1/2)

```
[1] Limpeza de objetos anteriores
  ok  : DROP TRIGGER TRG_AUD_PET
  ok  : DROP PROCEDURE PRC_FATURAMENTO_CLINICA
  ok  : DROP PROCEDURE PRC_ATENDIMENTOS_JSON
  ok  : DROP PROCEDURE PRC_IMPRIME_CLOB
  ok  : DROP FUNCTION FN_CALC_RISCO_PET
  ok  : DROP FUNCTION FN_MONTA_JSON
  ok  : DROP TABLE AUD_PET CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE LEITURA_SENSOR CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE ATENDIMENTO CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE PET CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE VETERINARIO CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE TUTOR CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE TIPO_SERVICO CASCADE CONSTRAINTS PURGE
  ok  : DROP TABLE CLINICA CASCADE CONSTRAINTS PURGE
  ok  : DROP SEQUENCE SEQ_PET
  ok  : DROP SEQUENCE SEQ_AUD_PET

PL/SQL procedure successfully completed.

Table created.

Table created.

Table created.

Table created.

Table created.

Table created.

Table created.

Table created.

Sequence created.

Sequence created.

Carga de dados concluida (CLINICA 6, TIPO_SERVICO 5, TUTOR 8, VETERINARIO 6, PET 10, ATENDIMENTO 34, LEITURA_SENSOR
      20).

Function created.

No errors.

Function created.

No errors.

Procedure created.

No errors.

Procedure created.

No errors.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 16 — Limpeza idempotente, carga e compilação dos objetos sem erros. (parte 1 de 2)

Saida do script | Compilacao dos objetos (SQL*Plus) (parte 2/2)

```
Procedure created.

No errors.

Trigger created.

No errors.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 16 — Limpeza idempotente, carga e compilação dos objetos sem erros. (parte 2 de 2)

Saida do script | 10.12 Registros por tabela (minimo exigido: 5)

```
TABELA          REGISTROS
--------------- ----------
CLINICA                  6
TIPO_SERVICO             5
TUTOR                    8
VETERINARIO              6
PET                     10
ATENDIMENTO             34
LEITURA_SENSOR          20
AUD_PET                  6

8 rows selected.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 17 — Registros por tabela após o roteiro de testes.

Saida do script | 10.13 Objetos do projeto e status de compilacao

```
OBJECT_TYPE  OBJECT_NAME                STATUS
------------ -------------------------- --------
FUNCTION     FN_CALC_RISCO_PET          VALID
FUNCTION     FN_MONTA_JSON              VALID
PROCEDURE    PRC_ATENDIMENTOS_JSON      VALID
PROCEDURE    PRC_FATURAMENTO_CLINICA    VALID
PROCEDURE    PRC_IMPRIME_CLOB           VALID
SEQUENCE     SEQ_AUD_PET                VALID
SEQUENCE     SEQ_PET                    VALID
TABLE        ATENDIMENTO                VALID
TABLE        AUD_PET                    VALID
TABLE        CLINICA                    VALID
TABLE        LEITURA_SENSOR             VALID
TABLE        PET                        VALID
TABLE        TIPO_SERVICO               VALID
TABLE        TUTOR                      VALID
TABLE        VETERINARIO                VALID
TRIGGER      TRG_AUD_PET                VALID

16 rows selected.
```

Conexao: clyvo@//localhost:1521/FREEPDB1 | Oracle Database 23ai Free | SQL*Plus | saida real do script

Print 18 — Objetos do projeto com status de compilação VALID.

11. Como executar e mapa de exceções

11.1 Execução

- Abrir **2TDSPF_2026_CodigoSql_Integrantes.sql** no SQL Developer (encoding UTF-8) e conectar na instância Oracle.
- Executar o arquivo inteiro como script (**F5**). O bloco [1] remove objetos de execuções anteriores, então o script pode ser reexecutado quantas vezes for necessário.
- Acompanhar a aba *Script Output*: cada bloco do roteiro de testes é precedido por um PROMPT ===== *10.x* =====.

11.2 Exceções por rotina e como cada uma é provocada

Rotina	Exceção	Como é provocada no roteiro	Bloco
FN_MONTA_JSON	e_qtd_divergente	2 chaves, 1 valor: ('id|nome', '1', 'N|S')	10.3
	e_tipo_invalido	tipo 'X' na lista de tipos	10.3
	VALUE_ERROR	tipo N com valor 'trinta'	10.3
	e_parametro_nulo	lista de chaves NULL	10.3
FN_CALC_RISCO_PET	NO_DATA_FOUND	pet 999 (inexistente)	10.5
	ZERO_DIVIDE	pet 10 (Zeus) sem leituras de sensor	10.5
	e_param_invalido	id -5	10.5
PRC_ATENDIMENTOS_JSON	NO_DATA_FOUND	clínica 999 (inexistente)	10.7
	e_sem_registros	clínica 6 (nova, sem atendimentos)	10.7
	e_parametro_invalido	id -1	10.7
	e_falha_conversao	defensiva: Função 1 devolveu NULL para uma linha	—
PRC_FATURAMENTO_CLINICA	VALUE_ERROR / INVALID_NUMBER	ano 'ABC'	10.9
	e_ano_invalido	ano '1990' (fora de 2000–2100)	10.9
	e_sem_movimento	ano '2031' (sem atendimentos)	10.9
TRG_AUD_PET	OTHERS → ORA-20051	qualquer falha ao gravar a auditoria desfaz a operação em PET	—
Todas	OTHERS	SQLCODE e SQLERRM impressos	—

11.3 Sobre a Sprint anterior

O código da 2ª Sprint não foi aproveitado nesta entrega: o projeto foi reconstruído do zero sobre o tema ClyvoVet, já incorporando integralmente os requisitos desta Sprint (conversão JSON manual, somatório manual com subtotais, tratamento de exceções demonstrável e auditoria por trigger).

Anexo A — Carga de dados completa

Seção [3] do script: todos os INSERTs, com a justificativa de cada conjunto de dados nos comentários.

```
-- ================================================================================
-- [3] CARGA DE DADOS
--     Todas as tabelas recebem pelo menos 5 registros. AUD_PET e populada pela trigger
--     durante o roteiro de testes (6 operacoes DML -> 6 registros).
-- ================================================================================
-- FEEDBACK OFF evita ~100 linhas de "1 row created."; a carga e conferida no bloco 10.12
SET FEEDBACK OFF
-- 3.1 CLINICA (6 registros). A unidade 6 e nova e ainda nao possui atendimentos:
--     ela e usada para demonstrar a excecao "sem registros" do Procedimento 1.
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (1, 'ClyvoVet Paulista',  'Sao Paulo', 'Bela
       Vista', '(11) 3000-1001');
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (2, 'ClyvoVet Pinheiros', 'Sao Paulo',
       'Pinheiros',  '(11) 3000-1002');
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (3, 'ClyvoVet Moema',     'Sao Paulo',
       'Moema',      '(11) 3000-1003');
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (4, 'ClyvoVet Santana',   'Sao Paulo',
       'Santana',    '(11) 3000-1004');
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (5, 'ClyvoVet Tatuape',   'Sao Paulo',
       'Tatuape',    '(11) 3000-1005');
INSERT INTO CLINICA (id_clinica, nome, cidade, bairro, telefone) VALUES (6, 'ClyvoVet Campinas',  'Campinas',
       'Cambui',     '(19) 3000-1006');
-- 3.2 TIPO_SERVICO (5 registros)
INSERT INTO TIPO_SERVICO (id_tipo_servico, descricao, preco_base) VALUES (1, 'Consulta clinica',      180.00);
INSERT INTO TIPO_SERVICO (id_tipo_servico, descricao, preco_base) VALUES (2, 'Vacinacao',             120.00);
INSERT INTO TIPO_SERVICO (id_tipo_servico, descricao, preco_base) VALUES (3, 'Exames laboratoriais',  250.00);
INSERT INTO TIPO_SERVICO (id_tipo_servico, descricao, preco_base) VALUES (4, 'Cirurgia',            1500.00);
INSERT INTO TIPO_SERVICO (id_tipo_servico, descricao, preco_base) VALUES (5, 'Ultrassonografia',      320.00);
-- 3.3 TUTOR (8 registros) - dados ficticios
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (1, 'Ana Paula Ribeiro',
       '111.222.333-01', 'ana.ribeiro@email.com',     '(11) 98888-1001', 'Sao Paulo');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (2, 'Bruno Carvalho Lima',
       '111.222.333-02', 'bruno.lima@email.com',      '(11) 98888-1002', 'Sao Paulo');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (3, 'Carla Mendes Souza',
       '111.222.333-03', 'carla.souza@email.com',     '(11) 98888-1003', 'Guarulhos');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (4, 'Diego Fernandes Alves',
       '111.222.333-04', 'diego.alves@email.com',     '(11) 98888-1004', 'Sao Paulo');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (5, 'Elaine Rocha Martins',
       '111.222.333-05', 'elaine.martins@email.com',  '(11) 98888-1005', 'Osasco');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (6, 'Fabio Nogueira Prado',
       '111.222.333-06', 'fabio.prado@email.com',     '(11) 98888-1006', 'Sao Paulo');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (7, 'Gabriela Santos Teixeira',
       '111.222.333-07', 'gabriela.teixeira@email.com','(11) 98888-1007', 'Santo Andre');
INSERT INTO TUTOR (id_tutor, nome, cpf, email, telefone, cidade) VALUES (8, 'Henrique Duarte Campos',
       '111.222.333-08', 'henrique.campos@email.com', '(11) 98888-1008', 'Sao Paulo');
-- 3.4 VETERINARIO (6 registros)
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (1, 'Dra. Mariana Costa',    'CRMV-SP
       12345', 'Clinica geral');
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (2, 'Dr. Rafael Almeida',    'CRMV-SP
       23456', 'Cirurgia');
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (3, 'Dra. Juliana Ferreira', 'CRMV-SP
       34567', 'Dermatologia');
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (4, 'Dr. Thiago Barbosa',    'CRMV-SP
       45678', 'Cardiologia');
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (5, 'Dra. Patricia Gomes',   'CRMV-SP
       56789', 'Diagnostico por imagem');
INSERT INTO VETERINARIO (id_veterinario, nome, crmv, especialidade) VALUES (6, 'Dr. Lucas Moreira',     'CRMV-SP
       67890', 'Clinica geral');
-- 3.5 PET (10 registros). Os pets 8, 9 e 10 nao possuem leituras de sensor:
--     o pet 10 (Zeus) demonstra a excecao ZERO_DIVIDE da Funcao 2.
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 1, 1, 'Rex',   'CAO',
         'Labrador',         'M', TO_DATE('12/03/2020', 'DD/MM/YYYY'), 32.50);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 2, 2, 'Mel',   'CAO',
         'Golden Retriever', 'F', TO_DATE('05/07/2019', 'DD/MM/YYYY'), 28.00);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 3, 3, 'Luna',
       'GATO', 'Siames',           'F', TO_DATE('20/11/2021', 'DD/MM/YYYY'),  4.20);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 4, 4, 'Thor',  'CAO',
         'Pastor Alemao',    'M', TO_DATE('15/01/2018', 'DD/MM/YYYY'), 38.00);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 5, 5, 'Nina',
       'GATO', 'Persa',            'F', TO_DATE('08/09/2022', 'DD/MM/YYYY'),  3.80);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 6, 6, 'Bob',   'CAO',
         'Beagle',           'M', TO_DATE('30/04/2021', 'DD/MM/YYYY'), 14.50);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 7, 7, 'Mia',
       'GATO', 'SRD',              'F', TO_DATE('12/12/2020', 'DD/MM/YYYY'),  4.50);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 8, 8, 'Simba',
```

```
        'GATO', 'Maine Coon',       'M', TO_DATE('25/06/2019', 'DD/MM/YYYY'),  7.80);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES ( 9, 1, 'Lola',  'CAO',
          'Poodle',           'F', TO_DATE('03/02/2023', 'DD/MM/YYYY'),  6.20);
INSERT INTO PET (id_pet, id_tutor, nome, especie, raca, sexo, data_nascimento, peso_kg) VALUES (10, 4, 'Zeus',  'CAO',
          'Rottweiler',       'M', TO_DATE('17/08/2017', 'DD/MM/YYYY'), 45.00);
-- 3.6 ATENDIMENTO (34 registros)
--     * 30 REALIZADOS em 2026 cobrindo as 25 combinacoes (5 clinicas x 5 servicos) + 5 repeticoes,
--       para que o Procedimento 2 realmente acumule valores por combinacao;
--     * 2 REALIZADOS em 2025 (mostra o filtro por ano);
--     * 2 AGENDADOS em 2026 (excluidos do faturamento pelo filtro de status).
-- Clinica 1 - Paulista
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 1,  1, 1, 1, 1, TO_DATE('10/01/2026', 'DD/MM/YYYY'),  180.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 2,  2, 1, 1, 2, TO_DATE('12/01/2026', 'DD/MM/YYYY'),  120.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 3,  3, 3, 1, 3, TO_DATE('15/01/2026', 'DD/MM/YYYY'),  250.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 4,  4, 2, 1, 4, TO_DATE('20/01/2026', 'DD/MM/YYYY'), 1500.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 5,  5, 5, 1, 5, TO_DATE('22/01/2026', 'DD/MM/YYYY'),  320.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 6,  2, 1, 1, 1, TO_DATE('05/02/2026', 'DD/MM/YYYY'),  195.00, 'REALIZADO');
-- Clinica 2 - Pinheiros
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 7,  6, 6, 2, 1, TO_DATE('03/02/2026', 'DD/MM/YYYY'),  180.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 8,  7, 6, 2, 2, TO_DATE('04/02/2026', 'DD/MM/YYYY'),  135.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES ( 9,  8, 3, 2, 3, TO_DATE('10/02/2026', 'DD/MM/YYYY'),  275.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (10,  9, 2, 2, 4, TO_DATE('14/02/2026', 'DD/MM/YYYY'), 1800.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (11, 10, 5, 2, 5, TO_DATE('18/02/2026', 'DD/MM/YYYY'),  320.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (12,  6, 6, 2, 2, TO_DATE('20/02/2026', 'DD/MM/YYYY'),  120.00, 'REALIZADO');
-- Clinica 3 - Moema
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (13,  1, 4, 3, 1, TO_DATE('02/03/2026', 'DD/MM/YYYY'),  200.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (14,  3, 1, 3, 2, TO_DATE('05/03/2026', 'DD/MM/YYYY'),  120.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (15,  5, 3, 3, 3, TO_DATE('09/03/2026', 'DD/MM/YYYY'),  250.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (16,  7, 2, 3, 4, TO_DATE('12/03/2026', 'DD/MM/YYYY'), 1650.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (17,  9, 5, 3, 5, TO_DATE('16/03/2026', 'DD/MM/YYYY'),  350.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (18,  1, 5, 3, 5, TO_DATE('20/03/2026', 'DD/MM/YYYY'),  320.00, 'REALIZADO');
-- Clinica 4 - Santana
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (19,  2, 6, 4, 1, TO_DATE('01/04/2026', 'DD/MM/YYYY'),  180.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (20,  4, 1, 4, 2, TO_DATE('03/04/2026', 'DD/MM/YYYY'),  120.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (21,  6, 4, 4, 3, TO_DATE('07/04/2026', 'DD/MM/YYYY'),  260.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (22,  8, 2, 4, 4, TO_DATE('10/04/2026', 'DD/MM/YYYY'), 2100.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (23, 10, 5, 4, 5, TO_DATE('14/04/2026', 'DD/MM/YYYY'),  320.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (24,  4, 4, 4, 3, TO_DATE('21/04/2026', 'DD/MM/YYYY'),  250.00, 'REALIZADO');
-- Clinica 5 - Tatuape
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (25,  3, 6, 5, 1, TO_DATE('04/05/2026', 'DD/MM/YYYY'),  180.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (26,  5, 6, 5, 2, TO_DATE('06/05/2026', 'DD/MM/YYYY'),  120.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (27,  7, 3, 5, 3, TO_DATE('11/05/2026', 'DD/MM/YYYY'),  250.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (28,  9, 2, 5, 4, TO_DATE('15/05/2026', 'DD/MM/YYYY'), 1500.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (29,  1, 5, 5, 5, TO_DATE('19/05/2026', 'DD/MM/YYYY'),  320.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (30, 10, 2, 5, 4, TO_DATE('26/05/2026', 'DD/MM/YYYY'), 1750.00, 'REALIZADO');
-- Historico de 2025 (fora do ano padrao dos testes)
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (31,  1, 1, 1, 1, TO_DATE('15/11/2025', 'DD/MM/YYYY'),  170.00, 'REALIZADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (32,  2, 1, 1, 2, TO_DATE('20/12/2025', 'DD/MM/YYYY'),  110.00, 'REALIZADO');
```

```
-- Agendados (nao entram no faturamento)
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (33,  8, 4, 2, 1, TO_DATE('15/10/2026', 'DD/MM/YYYY'),  180.00, 'AGENDADO');
INSERT INTO ATENDIMENTO (id_atendimento, id_pet, id_veterinario, id_clinica, id_tipo_servico, data_atendimento, valor,
         status) VALUES (34,  5, 5, 3, 5, TO_DATE('20/10/2026', 'DD/MM/YYYY'),  320.00, 'AGENDADO');
-- 3.7 LEITURA_SENSOR (20 registros). Cenarios reaproveitados do projeto IoT:
--     Rex saudavel, Mel com febre gradual (CRITICO), Luna com queda de SpO2 (ALTO),
--     Mia levemente febril (MODERADO), demais dentro da faixa (BAIXO).
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 1, 1, 'ESP32-P01', TO_DATE('01/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 38.4,  88, 98.0, 72.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 2, 1, 'ESP32-P01', TO_DATE('01/09/2026 14:00', 'DD/MM/YYYY HH24:MI'), 38.6,  92, 98.0, 68.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 3, 1, 'ESP32-P01', TO_DATE('01/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 38.5,  90, 98.0, 70.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 4, 2, 'ESP32-P02', TO_DATE('02/09/2026 06:00', 'DD/MM/YYYY HH24:MI'), 39.9, 150, 93.0, 20.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 5, 2, 'ESP32-P02', TO_DATE('02/09/2026 12:00', 'DD/MM/YYYY HH24:MI'), 40.2, 160, 92.0, 14.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 6, 2, 'ESP32-P02', TO_DATE('02/09/2026 18:00', 'DD/MM/YYYY HH24:MI'), 40.5, 170, 90.0,  8.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 7, 2, 'ESP32-P02', TO_DATE('03/09/2026 00:00', 'DD/MM/YYYY HH24:MI'), 40.6, 180, 89.0,  6.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 8, 3, 'ESP32-P03', TO_DATE('03/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 39.8, 230, 90.0,  8.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES ( 9, 3, 'ESP32-P03', TO_DATE('03/09/2026 14:00', 'DD/MM/YYYY HH24:MI'), 39.9, 235, 89.0,  5.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (10, 3, 'ESP32-P03', TO_DATE('03/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 40.0, 240, 88.0,  2.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (11, 4, 'ESP32-P04', TO_DATE('04/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 39.3, 100, 97.0, 55.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (12, 4, 'ESP32-P04', TO_DATE('04/09/2026 14:00', 'DD/MM/YYYY HH24:MI'), 39.4, 110, 97.0, 60.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (13, 4, 'ESP32-P04', TO_DATE('04/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 39.5, 120, 97.0, 65.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (14, 5, 'ESP32-P05', TO_DATE('05/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 38.6, 150, 98.0, 45.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (15, 5, 'ESP32-P05', TO_DATE('05/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 38.8, 160, 98.0, 55.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (16, 6, 'ESP32-P06', TO_DATE('06/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 38.8, 145, 96.0, 30.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (17, 6, 'ESP32-P06', TO_DATE('06/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 39.0, 155, 96.0, 40.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (18, 7, 'ESP32-P07', TO_DATE('07/09/2026 08:00', 'DD/MM/YYYY HH24:MI'), 39.9, 170, 94.0, 35.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (19, 7, 'ESP32-P07', TO_DATE('07/09/2026 14:00', 'DD/MM/YYYY HH24:MI'), 40.0, 180, 93.0, 40.0);
INSERT INTO LEITURA_SENSOR (id_leitura, id_pet, dispositivo, data_hora, temperatura, freq_cardiaca, spo2, atividade)
        VALUES (20, 7, 'ESP32-P07', TO_DATE('07/09/2026 20:00', 'DD/MM/YYYY HH24:MI'), 40.1, 190, 92.0, 45.0);
COMMIT;
SET FEEDBACK ON
PROMPT Carga de dados concluida (CLINICA 6, TIPO_SERVICO 5, TUTOR 8, VETERINARIO 6, PET 10, ATENDIMENTO 34,
        LEITURA_SENSOR 20).
```

Anexo B — Configuração da sessão e limpeza idempotente

Seções [0] e [1] do script e o cabeçalho com a ordem de execução.

```
-- ================================================================================
--  FIAP | 2TDSPF | 2026 | Mastering Relational and Non-Relational Database - Sprint 3
--  Projeto ClyvoVet - plataforma de saude digital para pets (clinicas, tutores,
--  veterinarios, atendimentos e telemetria IoT de sensores vestiveis).
--
--  Integrantes (ordem alfabetica):
--    Carlos Alberto Guedes Neto ........ RM 566022
--    Eduardo Novaes Mollo .............. RM 561515
--    Luan Peixoto Marins Rocha ......... RM 562258
--    Mathaus Victor Souza Marcelino .... RM 564146
--    Vinicius Luis Exposito Morassi .... RM 563340
--
--  ORDEM DE EXECUCAO (o script roda de cima para baixo, de uma vez, com F5 no SQL Developer):
--    [0] Configuracao da sessao
--    [1] Limpeza idempotente (permite reexecutar o script sem erros)
--    [2] Estrutura: tabelas, constraints e sequences
--    [3] Carga de dados (minimo de 5 registros por tabela)
--    [4] FUNCAO 1     - FN_MONTA_JSON            relacional -> JSON com conversao 100% manual
--    [5] FUNCAO 2     - FN_CALC_RISCO_PET        regra de negocio: score de risco clinico do pet
--    [6] UTILITARIO   - PRC_IMPRIME_CLOB         imprime um CLOB linha a linha (limite do DBMS_OUTPUT)
--    [7] PROCEDURE 1  - PRC_ATENDIMENTOS_JSON    JOIN de 6 tabelas -> array JSON (usa a Funcao 1)
--    [8] PROCEDURE 2  - PRC_FATURAMENTO_CLINICA  subtotais e total geral com somatorio manual
--    [9] AUDITORIA    - TRG_AUD_PET              AFTER INSERT OR UPDATE OR DELETE ON PET (usa a Funcao 1)
--   [10] ROTEIRO DE TESTES - execucoes normais, excecoes tratadas e conferencia da auditoria
--
--  REGRAS RESPEITADAS:
--    * Nenhuma funcao built-in de JSON (JSON_OBJECT, JSON_ARRAY, JSON_VALUE, JSON_QUERY,
--      JSON_TABLE, TO_JSON, JSON_SERIALIZE, APEX_JSON, DBMS_JSON...). Toda conversao e manual.
--    * Nenhum ROLLUP / CUBE / GROUPING SETS / GROUPING. Nas rotinas exigidas tambem nao ha
--      SUM / AVG / GROUP BY: todo somatorio e feito com acumuladores dentro do PL/SQL.
--    * Cada funcao/procedure trata no minimo 3 excecoes distintas + WHEN OTHERS, e o roteiro
--      de testes provoca cada uma delas para gerar os prints da documentacao.
--    * Literais de texto sem acentos, para o script rodar identico em qualquer charset de
--      banco/cliente. Os comentarios estao em portugues normal.
-- ================================================================================
-- ================================================================================
-- [0] CONFIGURACAO DA SESSAO
-- ================================================================================
-- (comandos SET nao aceitam comentario na mesma linha; por isso os comentarios vem antes)
-- FORMAT WRAPPED preserva os espacos a esquerda (alinhamento do relatorio do Procedimento 2)
SET SERVEROUTPUT ON SIZE UNLIMITED FORMAT WRAPPED
-- '&' nao e tratado como variavel de substituicao
SET DEFINE OFF
-- linhas JSON longas nao sao quebradas pelo cliente
SET LINESIZE 1000
SET PAGESIZE 200
-- alinhamento com espacos, nunca com tabulacoes
SET TAB OFF
SET FEEDBACK ON
-- exibe o conteudo completo das colunas CLOB nas consultas de conferencia
SET LONG 4000
SET LONGCHUNKSIZE 4000
-- ================================================================================
-- [1] LIMPEZA IDEMPOTENTE
--     Remove os objetos de execuções anteriores. Só ignora os erros "objeto não existe"
--     (ORA-00942 tabela, ORA-02289 sequence, ORA-04080 trigger, ORA-04043 procedure/function);
--     qualquer outro erro é propagado.
-- ================================================================================
DECLARE
    PROCEDURE remover(p_comando VARCHAR2) IS
    BEGIN
        EXECUTE IMMEDIATE p_comando;
        DBMS_OUTPUT.PUT_LINE('  ok  : ' || p_comando);
    EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE IN (-942, -2289, -4080, -4043) THEN
                DBMS_OUTPUT.PUT_LINE('  pula: ' || p_comando || ' (objeto nao existia)');
            ELSE
                RAISE;
            END IF;
    END remover;
BEGIN
    DBMS_OUTPUT.PUT_LINE('[1] Limpeza de objetos anteriores');
    remover('DROP TRIGGER TRG_AUD_PET');
```

```
    remover('DROP PROCEDURE PRC_FATURAMENTO_CLINICA');
    remover('DROP PROCEDURE PRC_ATENDIMENTOS_JSON');
    remover('DROP PROCEDURE PRC_IMPRIME_CLOB');
    remover('DROP FUNCTION FN_CALC_RISCO_PET');
    remover('DROP FUNCTION FN_MONTA_JSON');
    remover('DROP TABLE AUD_PET CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE LEITURA_SENSOR CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE ATENDIMENTO CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE PET CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE VETERINARIO CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE TUTOR CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE TIPO_SERVICO CASCADE CONSTRAINTS PURGE');
    remover('DROP TABLE CLINICA CASCADE CONSTRAINTS PURGE');
    remover('DROP SEQUENCE SEQ_PET');
    remover('DROP SEQUENCE SEQ_AUD_PET');
END;
/
```